Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTMENT OF SYSTEMS AND COMPUTER ENGINEERING

Protein Classification and 2D Structure Prediction using Deep Reinforcement Learning

Bachelor Final Project in European Computer Science

Author

Arbi Golemi

Supervisor

Carlos Manuel Jorge da Silva Pereira

Coimbra, May 2026

INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Resumo

A relação entre a sequência de aminoácidos de uma proteína e a sua estrutura tridimensional e função biológica é um foco central da biologia computacional moderna. Este projeto apresenta o desenvolvimento de uma plataforma web abrangente, construída com a framework Django, concebida para prever tanto a estrutura como as características funcionais das proteínas diretamente a partir de sequências FASTA. Para a anotação funcional, a plataforma integra a API DeepGO para prever os termos da Gene Ontology (GO), oferecendo um mapeamento rápido e acessível de sequência para função. Para abordar o complexo problema do enovelamento de proteínas, o estudo utiliza o modelo de rede 2D Hidrofóbico-Polar (HP). Um conjunto de algoritmos clássicos de otimização heurística incluindo Hill Climbing, Simulated Annealing e Replica Exchange Monte Carlo foi implementado e avaliado para estabelecer bases de referência estruturais. Com base nestes métodos clássicos, foram introduzidas técnicas de Inteligência Artificial para mapear trajetórias de enovelamento. A progressão do Tabular Q-Learning para uma Deep Q-Network (DQN) personalizada, baseada em PyTorch, demonstra a capacidade das Redes Neurais Convolucionais (CNNs) para aprender dependências espaciais dentro da grelha da proteína, superando as limitações dimensionais dos métodos tabulares. A plataforma resultante unifica estas diversas metodologias computacionais numa única interface, demonstrando a eficácia do Deep Reinforcement Learning na resolução de problemas complexos de otimização biofísica.

Palavras-chave: Enovelamento de Proteínas, Deep Reinforcement Learning, Django, Gene Ontology, Modelo HP.

Abstract

The relationship between a protein's amino acid sequence and its three-dimensional structure and biological function is a central focus of modern computational biology. This project presents the development of a comprehensive web-based platform, built with the Django framework, designed to predict both the structure and the functional characteristics of proteins directly from FASTA sequences. For functional annotation, the platform integrates the DeepGO API to predict Gene Ontology (GO) terms, offering rapid and accessible sequence-to-function mapping. To address the complex protein folding problem, the study utilizes the 2D Hydrophobic-Polar (HP) lattice model. A suite of classical heuristic optimization algorithms including Hill Climbing, Simulated Annealing, and Replica Exchange Monte Carlo was implemented and benchmarked to establish structural baselines. Building upon these classical methods, Artificial Intelligence techniques were introduced to map folding trajectories. The progression from Tabular Q-Learning to a custom PyTorch-based Deep Q-Network (DQN) demonstrates the capability of Convolutional Neural Networks (CNNs) to learn spatial dependencies within the protein grid, overcoming the dimensional limitations of tabular methods. The resulting platform unifies these diverse computational methodologies into a single interface, demonstrating the effectiveness of Deep Reinforcement Learning in solving complex biophysical optimization problems.

Keywords: Protein Folding, Deep Reinforcement Learning, Django, Gene Ontology, HP Model.

Epigraph

Nothing in life is to be feared, it is only to be understood. Now is the time to understand more, so that we may fear less.
Marie Curie

Dedication

To my parents, who sacrificed so much of their lives to give me a better one. This achievement is for you.

Ai miei genitori, che hanno sacrificato tanto della loro vita per offrirmene una migliore. Questo traguardo è per loro.

Prindërve të mi, që sakrifikuan aq shumë nga jeta e tyre për të më dhënë mua një të ardhme më të mirë. Ky sukses është për ta.

Acknowledgements

I would first like to thank my supervisor for their availability, support, and valuable advice throughout the development of this work.

I would like to thank Ca' Foscari University of Venice for the education I have received over the years and for giving me the opportunity to enrich my academic and personal journey. I would also like to thank all the professors who, through their teaching, contributed to my growth.

I would also like to thank the Polytechnic Institute of Coimbra and all the professors who, during this year, welcomed and guided me, contributing to my personal and professional development.

A special thanks goes to my family, especially my parents and my brother, for their love, sacrifices, and constant support, which allowed me to reach this milestone.

Finally, I would like to thank myself for the determination, patience, and strength to never give up in the face of difficulties.

Contents

List of Tables

List of Figures

List of Abbreviations

2D Two-dimensional

3D Three-dimensional

AA Amino acid

AI Artificial Intelligence

API Application Programming Interface

CNN Convolutional Neural Network

CPU Central Processing Unit

cryo-EM Cryogenic Electron Microscopy

CSS Cascading Style Sheets

CUDA Compute Unified Device Architecture

DB Database

DQN Deep Q-Network

DRL Deep Reinforcement Learning

FASTA FAST-All sequence format

GO Gene Ontology

GO:BP Gene Ontology: Biological Process

GO:CC Gene Ontology: Cellular Component

GO:MF Gene Ontology: Molecular Function

GET HTTP GET request method

GPU Graphics Processing Unit

H Hydrophobic residue

HC Hill Climbing

HP Hydrophobic-Polar

HTML HyperText Markup Language

IEEE *Institute of Electrical and Electronics Engineers*

ISEC Instituto Superior de Engenharia de Coimbra

JSON JavaScript Object Notation

MC Monte Carlo

MDP Markov Decision Process

MSE Mean Squared Error

MVT Model-View-Template

NP Nondeterministic Polynomial time

P Polar residue

PDB Protein Data Bank

POST HTTP POST request method

PPO Proximal Policy Optimization

QL Q-Learning

REMC Replica Exchange Monte Carlo

RL Reinforcement Learning

RMSD Root Mean Square Deviation

SA Simulated Annealing

SAC Soft Actor-Critic

SAW Self-Avoiding Walk

SVG Scalable Vector Graphics

TD Temporal Difference

TM-score Template Modelling score

UniProt Universal Protein Resource

List of Symbols

a Action selected by the reinforcement learning agent

a' Candidate or next action

$\mathcal{A}$ Action space

β_k Inverse temperature of replica k, defined as $\beta_k = 1/T_k$

$\mathcal{C}$ Protein conformation represented as lattice coordinates

$\mathcal{C}'$ Candidate conformation after applying a move

$\mathcal{D}$ Replay buffer used during DQN training

$d(p_i, p_j)$ Manhattan distance between residues i and j

ΔE Energy variation, $E(\mathcal{C}') - E(\mathcal{C})$

$E(\mathcal{C})$ HP energy of conformation $\mathcal{C}$

$E_{\textbf{best}}$ Best energy found during a search or training run

ε Exploration probability in an ε-greedy policy

ε_0 Initial exploration probability

$\varepsilon_{\min}$ Minimum exploration probability

$g_{\textbf{channel}}(x, y)$ Value of the DQN input grid at position (x, y) for one residue channel

γ Discount factor for future rewards

i, j Residue indices in the protein sequence

$\mathbb{1}(\cdot)$ Indicator function, equal to 1 when the condition is true and 0 otherwise

k Pivot index or replica index, depending on context

$\mathcal{L}(\theta)$ DQN loss function

m Move sampled from the classical move set

N Number of replicas in Replica Exchange Monte Carlo

n Protein sequence length

p_i Lattice position of residue i

$P(\textbf{accept})$ Probability of accepting a candidate move

$P(\textbf{swap})$ Probability of swapping two neighbouring REMC replicas

$\mathcal{P}$ State transition probability function in the MDP formulation

$Q(s, a)$ Q-value for taking action a in state s

$Q(s, a; \theta)$ Deep Q-Network approximation of the Q-value using parameters θ

$Q(s, a; \theta^-)$ Target-network Q-value using parameters θ^-

$R(s, a)$ Reward obtained after taking action a in state s

r Immediate reward in a transition

s Current state

s' Next state after applying an action

$\mathcal{S}$ State space

t Iteration, step, or episode index

T Temperature parameter used by Monte Carlo and Simulated Annealing

T_0 Initial temperature

T_f Final temperature

T_i Temperature at iteration i

T_k Temperature assigned to replica k

$T_{\max}$ Maximum number of iterations or search steps

$T_{\textbf{decay}}$ Number of steps used for exploration decay

θ Trainable parameters of the policy network

θ^- Parameters of the target network

x_i, y_i Two-dimensional lattice coordinates of residue i

y Temporal-difference target used in DQN updates

$\mathbb{Z}^2$ Two-dimensional integer lattice

α Learning rate in Tabular Q-Learning

η Neural-network learning rate used by the Adam optimizer

1 Introduction

Proteins are the fundamental building blocks of life, executing a vast array of functions within living organisms, ranging from catalyzing metabolic reactions and DNA replication to providing structural support and transporting molecules. The specific function of a protein is inextricably linked to its three-dimensional (3D) structure, which is uniquely determined by its one-dimensional sequence of amino acids. Understanding how this linear sequence spontaneously folds into a complex, functional 3D conformation remains one of the most significant challenges in computational biology and bioinformatics widely known as the **protein folding problem**.

The importance of solving this problem cannot be overstated. Misfolded proteins are directly implicated in a range of severe diseases, including Alzheimer's disease, Parkinson's disease, and various cancers. Conversely, successfully predicting a protein's native structure from its sequence would accelerate drug discovery, enzyme engineering, and the design of novel biomaterials. The exponential growth of sequencing data driven by advances such as next-generation sequencing has created a vast *sequence-structure gap*: millions of protein sequences have been deposited in databases like UniProt [1], yet experimentally determined 3D structures (deposited in the Protein Data Bank, PDB [2]) remain orders of magnitude fewer.

To fully comprehend the scope of this project, several foundational concepts must be introduced:

- **The Protein Folding Problem:** The challenge of computationally predicting the native, energy-minimized 3D structure of a protein based solely on its amino acid sequence. Solving it with high accuracy, as demonstrated by AlphaFold2 [3], represents a landmark achievement of modern computational biology.
- **The HP (Hydrophobic-Polar) Model:** A simplified lattice-based model used in computational biology to study protein folding. It abstracts the 20 amino acids into two types Hydrophobic (H) and Polar (P) capturing the dominant driving force of folding: the burial of hydrophobic residues away from the aqueous solvent [4]. Despite its simplicity, finding the minimum-energy configuration in the HP model is NP-complete [5].
- **Heuristic Optimization Algorithms:** Classical computational techniques (Hill Climbing, Simulated Annealing, Monte Carlo methods) used to find approximate solutions to computationally intractable energy minimization problems by stochastic exploration of the conformation space [6, 7, 8].

- **Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL):** A branch of machine learning where an agent learns optimal decision-making by interacting with an environment to maximize cumulative reward [9, 10]. Deep Q-Networks (DQN), which use Convolutional Neural Networks as function approximators, extend this paradigm to handle the high-dimensional state spaces arising from realistic protein sequences [11].
- **Gene Ontology (GO) and Function Prediction:** A standardized vocabulary describing the biological functions of gene products, organized into three domains: Biological Process, Molecular Function, and Cellular Component [12]. Tools such as DeepGO [13] use deep learning to predict GO terms directly from protein sequences.
- **3D Structure Retrieval:** State-of-the-art systems such as AlphaFold [3] and ESMFold [14] can predict full-atom 3D protein structures at near-experimental accuracy and expose their predictions through publicly accessible APIs.

1.1 Vision and scope

Traditionally, determining the structure of a protein relies on experimental techniques such as X-ray crystallography and cryogenic electron microscopy (cryo-EM). While highly accurate, these methods are exceptionally resource-intensive, expensive, and time-consuming. The growing sequence-structure gap necessitates the development of efficient computational methods capable of predicting protein structures and functions directly from amino acid sequences.

The primary vision of this project is to bridge this gap by developing a comprehensive, user-friendly web application built using the Django framework [15] that integrates multiple computational methods for both protein structure and function prediction. The system is designed to be accessible to researchers and students without requiring local computational infrastructure.

Specifically, the scope of the project focuses on two main pillars:

1. **2D Structure Prediction (HP Model):** The implementation, benchmarking, and comparison of a progressive suite of algorithms to solve the 2D HP protein folding problem. This includes classical stochastic optimization methods Hill Climbing (HC), Simulated Annealing (SA), Monte Carlo (MC), and Replica Exchange Monte Carlo (REMC) alongside modern Artificial Intelligence approaches: Tabular Q-Learning and a custom Convolutional Deep Q-Network (DQN) implemented in PyTorch [16].
2. **Function Prediction and 3D Integration:** The integration of state-of-the-art Deep Learning APIs specifically DeepGO to predict functional Gene Ontology (GO)

terms directly from FASTA sequences. Additionally, the system serves as a bridge to 3D structure prediction tools (AlphaFold DB [17] and ESMFold), enabling users to retrieve and visualize known or predicted 3D structures without requiring local installations of these computationally demanding models.

1.2 Project schedule

The development of this project was organized into three structured phases, ensuring a systematic and incremental approach:

- **Phase 1 — Web Application and Function Prediction:** The initial phase focused on developing the core Django web architecture, designing the user interface, and integrating the DeepGO external API for Gene Ontology functional annotations. This phase established the basic sequence processing pipeline and the FASTA parsing utilities.
- **Phase 2 — Classical Optimization Implementation:** This phase involved the design and benchmarking of classical heuristic algorithms (HC, SA, MC, REMC) to solve the 2D HP folding problem. A shared move set (end-flip, kink-jump, crankshaft, pivot) was implemented and validated, and a comparative benchmarking study was conducted across 20 real UniProt protein sequences of 56–98 amino acids.
- **Phase 3 — Reinforcement Learning Integration:** The final phase introduced AI-based approaches: first Tabular Q-Learning (suitable for short sequences) and then a custom CNN-based Deep Q-Network (DQN) implemented in PyTorch. The final DQN uses a constructive formulation with a local $2 \times 21 \times 21$ rotation-invariant grid representation and relative directional actions. The DQN was trained and validated against the classical baseline methods.

1.3 Organization of the report

The remainder of this document is organized as follows:

Chapter 2 — Theoretical Background: Provides a formal review of the HP model, the neighbour and energy definitions, the shared move set, and each optimization algorithm (HC, SA, MC/REMC, Tabular Q-Learning, DQN), including their mathematical formulations and pseudocode.

Chapter 3 — Web Platform: Details the requirement analysis, user interface prototype, technical implementation of the Django web application, and the testing procedures for the integrated system.

Chapter 4 — Algorithm Development: Describes the full 2D structure prediction pi-

peline, the specific implementation details of all algorithms, the DQN architecture and training configuration, and a comparative analysis of the benchmarking results across all methods.

Chapter 5 — Conclusion and Future Work: Summarizes the main achievements of the project, discusses current limitations, and outlines potential avenues for future development and research.

2 Deep learning for 2D structure prediction

This chapter provides the theoretical foundation necessary to understand the computational approaches utilized in this project. It begins with a brief biological background on proteins and the folding problem, subsequently introducing the simplified lattice model used to represent them computationally. The chapter then explores classical optimization methods explaining each one individually before delving into the core artificial intelligence techniques, namely Reinforcement Learning and Deep Reinforcement Learning, that form the basis of the predictive models developed in this work.

2.1 Proteins

Proteins are large, complex macromolecules that play many critical roles in the body. They do most of the work in cells and are required for the structure, function, and regulation of the body's tissues and organs. A protein is constructed from a set of 20 standard amino acids, linked together in a linear chain known as the *primary structure*.

The functionality of a protein is highly dependent on its specific three-dimensional shape, which it spontaneously adopts through a process called **protein folding**. Understanding how a sequence of amino acids dictates the final 3D structure the so-called "Protein Folding Problem" is a major pursuit in computational biology.

To make the folding problem computationally tractable, researchers often use simplified lattice models. The most prominent of these is the **Hydrophobic-Polar (HP) model**, introduced by Ken Dill [4]. In the HP model, the 20 amino acids are abstracted into two classes based on their affinity for water:

- **Hydrophobic (H):** water-repelling amino acids (e.g. Alanine, Cysteine, Phenylalanine, Isoleucine, Leucine, Methionine, Valine, Tryptophan, Tyrosine).
- **Polar (P):** water-attracting amino acids (all remaining residues).

The folding process is driven by the hydrophobic effect, where H monomers tend to cluster together in the core of the structure to minimize exposure to the aqueous environment.

2.2 The HP lattice model

In a 2D square lattice HP model, a protein sequence of length n is embedded on a 2D integer grid. Each amino acid i occupies a distinct lattice site $(x_i, y_i) \in \mathbb{Z}^2$, and consecutive residues must occupy adjacent lattice sites (i.e. $\|p_i - p_{i+1}\|_1 = 1$). The chain must form a **self-avoiding walk** (SAW): no two residues can share the same lattice site.

2.2.1 Neighbour definition

Two residues i and j are defined as **topological neighbours** if:

1. They occupy adjacent grid sites, i.e. $|x_i - x_j| + |y_i - y_j| = 1$ (Manhattan distance of 1), **and**
2. They are **non-consecutive** in the primary sequence, i.e. $|i - j| > 1$.

This distinction is fundamental: consecutive residues are bonded by the peptide backbone (covalent bond), while non-consecutive residues that happen to be spatially adjacent form **topological contacts**. Only these non-covalent contacts contribute to the free energy of the conformation.

2.2.2 Energy function

The objective function to be minimized is the **HP energy**:

$$E(\mathcal{C}) = -\sum_{\substack{i<j \\ |i-j|>1}} \mathbb{1}[d(p_i, p_j) = 1] \cdot \mathbb{1}[\text{type}(i) = \text{H}] \cdot \mathbb{1}[\text{type}(j) = \text{H}] \tag{2.1}$$

where $\mathcal{C}$ denotes a conformation (a set of lattice positions), $d(p_i, p_j)$ is the Manhattan distance between residues i and j, and $\mathbb{1}[\cdot]$ is the indicator function. Each H-H topological contact contributes -1 to the total energy; a lower (more negative) energy indicates a more stable, compact structure.

Finding the global minimum of $E(\mathcal{C})$ in the HP model is known to be NP-complete [5]. Consequently, exact algorithms are computationally prohibitive for sequences longer than approximately 15 residues, necessitating the use of heuristic or learned optimization methods.

2.3 Move set

To explore the conformation space, all algorithms in this project use a shared set of **local moves** that alter the protein structure while preserving chain connectivity and the self-avoiding walk constraint:

- **End-flip:** The first or last residue of the chain is repositioned to one of the free lattice sites adjacent to its sole bonded neighbour. This move only affects a single terminal residue and is applicable at any time.
- **Kink-jump:** Applicable when an internal residue i forms a 90° bend (a "kink"), i.e. when $|x_{i-1} - x_{i+1}| = |y_{i-1} - y_{i+1}| = 1$. The residue is flipped to the diagonally opposite corner of the 2×2 square formed by p_{i-1}, p_i, and p_{i+1}.
- **Crankshaft:** Applicable when two adjacent internal residues i and $i+1$ form a U-shape, i.e. when $\|p_{i-1} - p_{i+2}\|_1 = 1$. Both residues are simultaneously rotated 180° around the axis defined by p_{i-1} and p_{i+2}.
- **Pivot:** A pivot point k is selected at random, and all residues from $k+1$ to $n-1$ are rotated by $\pm 90°$ or $180°$ around p_k. The move is accepted only if the resulting conformation is self-avoiding (i.e. all n positions remain distinct).

In the implementation, the four move types are sampled with weights $[0.2, 0.3, 0.3, 0.2]$ to favour the more local (and therefore more frequently valid) kink-jump and crankshaft moves over the more destructive pivot.

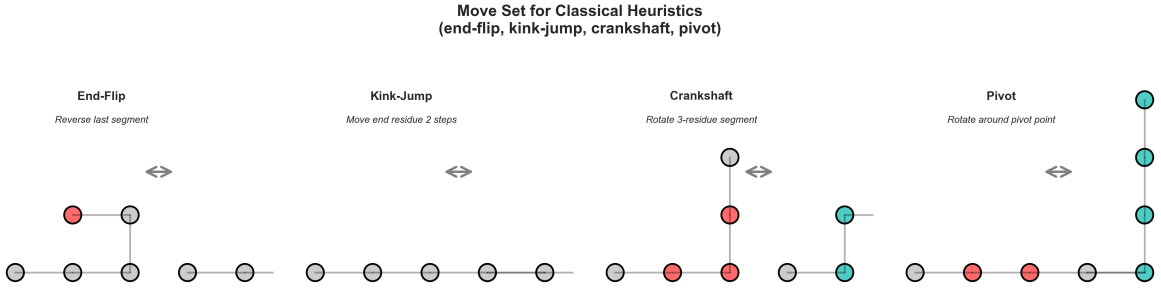

Figure 2.1: Move set used by the classical heuristics. The highlighted residues show the part of the chain affected by each local perturbation.

2.4 Classical optimization methods

Because exact enumeration is infeasible, three classical heuristic search algorithms were implemented and benchmarked as baselines.

2.4.1 Hill Climbing

Hill Climbing (HC) is a greedy local search algorithm. Starting from an initial straight-line conformation, it iteratively proposes a random move from the move set and **accepts it only if the new energy is non-increasing**:

$$\text{Accept if } E(\mathcal{C}') \leq E(\mathcal{C}) \tag{2.2}$$

The algorithm maintains a global best solution throughout the search. Although HC is computationally efficient and converges rapidly, it is highly susceptible to **local minima**: once the algorithm reaches a conformation from which no single move leads to an improvement, the search stalls.

Algorithm 1 Hill Climbing for HP folding

$\mathcal{C} \leftarrow$ straight-line conformation
$E_{\text{best}} \leftarrow E(\mathcal{C})$
for $t = 1$ **to** $T_{\max}$ **do**
    $m \sim$ Uniform({end-flip, kink-jump, crankshaft, pivot})
    $\mathcal{C}' \leftarrow \text{apply}(m, \mathcal{C})$ ▷ returns `None` if invalid
    if $\mathcal{C}' \neq$ `None` **and** $E(\mathcal{C}') \leq E(\mathcal{C})$ **then**
        $\mathcal{C} \leftarrow \mathcal{C}'$
        if $E(\mathcal{C}) < E_{\text{best}}$ **then**
            $E_{\text{best}} \leftarrow E(\mathcal{C})$; save best conformation
        end if
    end if
end for
return best conformation, E_{best}

2.4.2 Simulated Annealing

Simulated Annealing (SA), inspired by the metallurgical process of annealing, extends HC by occasionally accepting moves that *worsen* the energy [7]. This probabilistic acceptance criterion allows the search to escape local minima early in the process. The key idea is a temperature parameter T that decreases over time according to a **cooling schedule**.

A candidate move is accepted with probability:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{if } \Delta E > 0 \end{cases} \tag{2.3}$$

where $\Delta E = E(\mathcal{C}') - E(\mathcal{C})$. When the temperature T is high, even largely worse moves are accepted, promoting broad exploration. As $T \to 0$, the algorithm becomes equivalent to HC.

In this project, an **exponential cooling schedule** is used:

$$T_i = T_0 \cdot \left(\frac{T_f}{T_0}\right)^{i/(T_{\max}-1)} \tag{2.4}$$

where $T_0 = 10.0$ is the initial temperature, $T_f = 0.01$ is the final temperature, and i is

the current iteration index.

Algorithm 2 Simulated Annealing for HP folding

$\mathcal{C} \leftarrow$ straight-line conformation; $T \leftarrow T_0$
for $t = 1$ **to** $T_{\max}$ **do**
  $T \leftarrow T_0 \cdot (T_f/T_0)^{t/(T_{\max}-1)}$ ▷ exponential cooling
  $m \sim$ Uniform(move set); $\mathcal{C}' \leftarrow \text{apply}(m, \mathcal{C})$
  if $\mathcal{C}' \neq$ `None` **then**
    $\Delta E \leftarrow E(\mathcal{C}') - E(\mathcal{C})$
    if $\Delta E \leq 0$ **or** $\text{Uniform}(0,1) < e^{-\Delta E/T}$ **then**
      $\mathcal{C} \leftarrow \mathcal{C}'$
    end if
    if $E(\mathcal{C}) < E_{\text{best}}$ **then**
      $E_{\text{best}} \leftarrow E(\mathcal{C})$; save best conformation
    end if
  end if
end for
return best conformation, E_{best}

2.4.3 Monte Carlo and Replica Exchange Monte Carlo

Monte Carlo (MC) methods are closely related to SA but operate at a *constant* temperature, using the Metropolis criterion (Equation 2.3) to accept or reject moves [6]. Because the temperature does not change, MC is particularly useful for sampling the thermodynamic ensemble of conformations at a given temperature T.

Replica Exchange Monte Carlo (REMC), also known as parallel tempering, addresses the limitation of single-temperature MC by running N independent MC replicas in parallel, each at a different temperature $T_1 < T_2 < \cdots < T_N$ [8, 18]. Periodically, adjacent replicas k and $k+1$ attempt to **swap** their conformations with probability:

$$P(\text{swap}) = \min\big(1,\ e^{(\beta_k - \beta_{k+1})(E_k - E_{k+1})}\big) \tag{2.5}$$

where $\beta_k = 1/T_k$. This exchange mechanism allows low-temperature replicas (which exploit good solutions) to benefit from configurations discovered by high-temperature replicas (which explore broadly), vastly improving the sampling of the energy landscape.

2.5 Reinforcement learning

While heuristic algorithms search the energy landscape directly through stochastic perturbations, **Reinforcement Learning (RL)** approaches the problem differently by

training an *agent* to learn *how* to fold a protein through interaction with the environment [9].

2.5.1 Markov Decision Process formulation

The HP folding problem is formulated as a **Markov Decision Process (MDP)** defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$:

- **State space $\mathcal{S}$:** Each state $s \in \mathcal{S}$ represents the current conformation of the protein chain. In the tabular implementation, this is encoded as a canonical tuple of (x, y) positions, normalized for translation and one rotation symmetry, so that equivalent conformations map to the same state. In the DQN implementation (constructive approach), the state is a rotation-invariant 21×21 two-channel grid relative to the chain head and current direction.
- **Action space $\mathcal{A}$:** In the classical algorithms (HC, SA, MC, REMC), the agent chooses one of four move types: $\mathcal{A} = \{\text{end-flip}, \text{kink-jump}, \text{crankshaft}, \text{pivot}\}$. In the constructive DQN approach, actions represent relative directions for the next residue: $\mathcal{A} = \{\text{Forward}, \text{Left}, \text{Right}\}$, where rotations are applied to the current direction vector.
- **Reward function $\mathcal{R}$:** The reward received after taking action a in state s and transitioning to state s' is:

$$R(s,a) = \begin{cases} E(s) - E(s') & \text{if the move is valid } (= \Delta E > 0 \text{ means energy reduction}) \\ -1 & \text{if the move is invalid (collision / no valid position)} \end{cases} \tag{2.6}$$

  This reward is positive when the new conformation has a lower energy (more H-H contacts), zero when energy is unchanged, and negative for invalid moves. The agent therefore learns to prefer energy-reducing moves and to avoid collisions.
- **Discount factor $\gamma \in [0, 1)$:** Controls how much the agent values future rewards relative to immediate ones. The tabular implementation used $\gamma = 0.95$, while the DQN training run used $\gamma = 0.99$.

2.5.2 Tabular Q-Learning

Tabular Q-Learning is a model-free, off-policy RL algorithm [10]. The agent maintains a table (the *Q-table*) $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, where $Q(s, a)$ represents the expected cumulative discounted reward obtained by taking action a in state s and then following the optimal policy. Q-values are updated iteratively using the **Bellman equation**:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{2.7}$$

where $\alpha \in (0, 1]$ is the *learning rate* (set to 0.1) that controls how quickly new information overrides old estimates, and $\gamma = 0.95$ is the discount factor. The term $R(s, a) + \gamma \max_{a'} Q(s', a')$ is called the *TD target* (Temporal Difference target), and $\delta = \text{TD target} - Q(s, a)$ is the *TD error*.

Action selection follows an ε**-greedy policy**: with probability ε the agent selects a random action (exploration), and with probability $1-\varepsilon$ it selects the action with the highest Q-value (exploitation). The exploration rate ε is decayed linearly from 1.0 to 0.05 over the first half of training:

$$\varepsilon_t = \max\left(\varepsilon_{\min},\ \varepsilon_0 - (\varepsilon_0 - \varepsilon_{\min}) \cdot \frac{t}{\lfloor T_{\text{decay}} \rfloor}\right) \tag{2.8}$$

where $T_{\text{decay}} = \lfloor \text{episodes} \times \text{steps\_per\_episode}/2 \rfloor$.

Algorithm 3 Tabular Q-Learning for HP folding

Initialize $Q(s, a) \leftarrow 0$ for all s, a; $\varepsilon \leftarrow 1.0$
for each episode **do**
  $\mathcal{C} \leftarrow$ straight-line conformation; $s \leftarrow \text{encode}(\mathcal{C})$
  for each step **do**
    Decay ε according to Equation 2.8
    if $\text{Uniform}(0, 1) < \varepsilon$ **then**
      $a \leftarrow$ random action ▷ explore
    else
      $a \leftarrow \arg\max_{a'} Q(s, a')$ ▷ exploit
    end if
    $\mathcal{C}' \leftarrow \text{apply}(a, \mathcal{C})$
    Compute reward R via Equation 2.6
    $s' \leftarrow \text{encode}(\mathcal{C}')$
    $Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a''} Q(s', a'') - Q(s, a)]$
    $s \leftarrow s'$; $\mathcal{C} \leftarrow \mathcal{C}'$; update global best
  end for
end for
return best conformation, E_{best}, Q-table

Tabular Q-Learning is effective for short sequences (up to ≈ 25 residues) but suffers from the *curse of dimensionality*: the number of distinct conformations grows exponentially with chain length, making the Q-table intractably large for realistic proteins.

2.6 Deep reinforcement learning

Deep Reinforcement Learning (DRL) overcomes the scalability limitation by replacing the Q-table with a parametric function approximator: a **Deep Q-Network (DQN)** introduced by Mnih et al. [11]. Instead of storing one Q-value per state-action pair, the network $Q(s, a; \theta)$ with parameters θ maps a state representation directly to a vector of Q-values for all actions.

2.6.1 State representation

In the constructive DQN implementation, the state is encoded as a two-channel grid of size 21×21 (nine times smaller, focusing only on the local neighborhood). The encoding is rotation-invariant: the grid is centered on the chain's head (last positioned residue) and rotated so that the current direction vector always points upward. Each channel encodes residue type:

$$g_{\text{channel}}[x, y] = \begin{cases} 1.0 & \text{if a residue of the corresponding type occupies the rotated position } (x, y) \\ 0.0 & \text{(empty site)} \end{cases} \tag{2.9}$$

Channel 0 represents Hydrophobic (H) residues, channel 1 represents Polar (P) residues. This rotation-invariant encoding dramatically reduces state space size while ensuring that the CNN learns folding patterns that are independent of the chain's global position and orientation, enabling better generalization across proteins of different lengths and H-content.

2.6.2 Network architecture

The neural network is a **Convolutional Neural Network (CNN)** designed to process $2 \times 21 \times 21$ inputs. The final architecture used in the Colab-trained model consists of:

1. **Input:** Two-channel grid $(2 \times 21 \times 21)$ representing H and P residue positions.
2. **Convolutional block 1:** `Conv2d`$(2, 32$, kernel 3×3, padding $1) \rightarrow$ `ReLU`. Output: $32 \times 21 \times 21$.
3. **Convolutional block 2:** `Conv2d`$(32, 64$, kernel 3×3, padding $1) \rightarrow$ `ReLU`. Output: $64 \times 21 \times 21$.
4. **Convolutional block 3:** `Conv2d`$(64, 64$, kernel 3×3, padding $1) \rightarrow$ `ReLU`. Output: $64 \times 21 \times 21$.
5. **Flatten:** $64 \times 21 \times 21 = 28{,}224$ features.
6. **Fully connected:** `Linear`$(28224, 256) \rightarrow$ `ReLU` $\rightarrow$ `Linear`$(256, 3)$.

The output layer produces 3 Q-values, one per relative direction action (Forward, Left,

Right). The compact 21×21 grid dramatically reduces memory overhead while focusing on local patterns. Combined with the rotation-invariant encoding, this architecture enables the CNN to learn generalizable folding strategies that transfer effectively across proteins of different lengths and compositions.

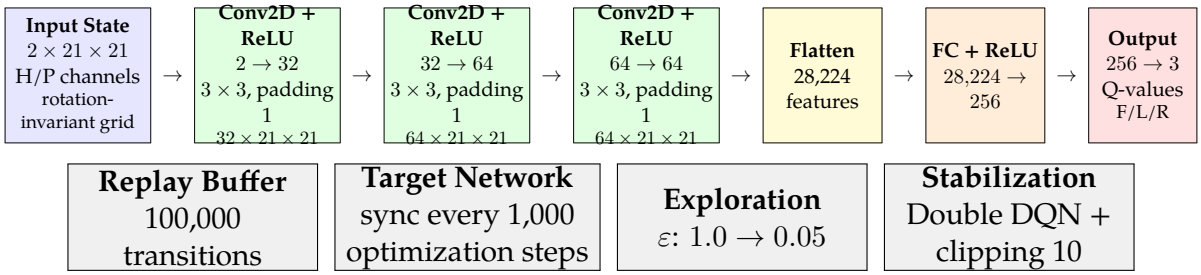

Figure 2.2: Constructive DQN architecture used for residue-by-residue folding. The two input channels encode hydrophobic and polar residues, and the output layer estimates Q-values for the relative actions Forward, Left and Right.

2.6.3 Training procedure

DQN training relies on key stabilization mechanisms:

- **Constructive Environment:** Unlike perturbative approaches that start with a straight line and modify it, the constructive environment builds the protein one amino acid at a time. The first two residues are positioned to define an initial direction; subsequent residues are placed via relative direction decisions (Forward/ Left/ Right). This guarantees self-avoiding walks by design and simplifies the state space.
- **Experience Replay:** Transitions (s, a, r, s') are stored in a *Replay Buffer* of capacity $100{,}000$. At each training step, a random mini-batch (batch size 128) is sampled. This breaks temporal correlations and reduces gradient variance.
- **Target Network:** A separate network copy, $Q(s, a; \theta^-)$, computes TD targets. Its weights θ^- are synchronized every 1,000 optimization steps, preventing oscillations from constantly moving targets.
- **Reward Structure:** $R = +1.0$ for each new H-H contact formed (direct energy feedback), -1.0 for invalid moves (collisions), $+0.5$ bonus for completing the full chain without collision, with additional shaping for low-H-density proteins.
- **Stability Mechanisms:** The training loop uses Double DQN target selection and gradient clipping with maximum norm 10.

The loss function is the **Mean Squared Error** (**MSE**) between predicted and target Q-values. In the final implementation, the target action is selected by the policy network

and evaluated by the target network, following the Double DQN formulation:

$$a^* = \arg\max_{a'} Q(s', a'; \theta) \quad ; \quad y = r + \gamma Q(s', a^*; \theta^-)(1 - done) \quad ; \quad \mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right] \tag{2.10}$$

Network parameters θ are optimized via **Adam** [19] with learning rate $\eta = 0.001$, $\gamma = 0.99$, and ε-decay from 1.0 to 0.05 over training. Training is performed on a dataset of 20 real UniProt proteins for 20,000 episodes, with GPU acceleration available.

Algorithm 4 Constructive Deep Q-Network for HP folding

Initialize policy network $Q(s, a; \theta)$ and target network $Q(s, a; \theta^-)$
Copy initial weights: $\theta^- \leftarrow \theta$
Initialize replay buffer $\mathcal{D}$ with capacity $100{,}000$
Set $\varepsilon \leftarrow 1.0$, $\varepsilon_{\min} \leftarrow 0.05$, $\gamma \leftarrow 0.99$
for episode $= 1$ to $20{,}000$ **do**
    Sample one HP sequence from the training set
    Reset constructive environment with first two residues fixed
    while the chain is not complete and no collision occurred **do**
        Encode partial chain as a $2 \times 21 \times 21$ rotation-invariant grid s
        if $\text{Uniform}(0, 1) < \varepsilon$ **then**
            Select random action $a \in \{\text{Forward}, \text{Left}, \text{Right}\}$
        else
            Select $a = \arg\max_{a'} Q(s, a'; \theta)$
        end if
        Apply a to place the next residue and observe $(r, s', done)$
        Store transition $(s, a, r, s', done)$ in $\mathcal{D}$
        if $|\mathcal{D}|$ is larger than the mini-batch size **then**
            Sample a random mini-batch from $\mathcal{D}$
            Select $a^* = \arg\max_{a'} Q(s', a'; \theta)$ and compute $y = r + \gamma Q(s', a^*; \theta^-)(1 - done)$
            Update θ by minimizing $(y - Q(s, a; \theta))^2$
        end if
    end while
    Decay $\varepsilon \leftarrow \max(\varepsilon_{\min}, 0.995\varepsilon)$
    if 1,000 optimization steps have elapsed **then**
        Synchronize target network: $\theta^- \leftarrow \theta$
    end if
end for
Save trained weights to `dqn_weights.pth`

2.7 Summary

This chapter detailed the theoretical concepts underlying the computational approaches to the 2D protein folding problem. We began by formalizing the HP lattice model, defining the notion of topological neighbours and the HP energy function. We then described the shared move set (end-flip, kink-jump, crankshaft, pivot) used by all algorithms. For the classical baseline methods, each algorithm was presented individually: Hill Climbing as a fast but locally trapped greedy search, Simulated Annealing as a temperature-controlled stochastic method capable of escaping local minima, and Replica Exchange Monte Carlo as a parallel, multi-temperature approach for robust landscape exploration. Finally, we introduced the Reinforcement Learning paradigm, formally defining the MDP formulation including the state space, action space, and reward function. We traced the evolution from Tabular Q-Learning which learns discrete folding policies via the Bellman equation and ε-greedy exploration to Deep Q-Networks, which replace the Q-table with a CNN capable of processing large spatial grids, offering a scalable method for predicting the structures of complex proteins.

3 Web platform

This chapter details the design, development, and testing of the interactive web platform created for this project. The platform serves as the primary user-facing interface, enabling seamless protein sequence analysis, functional annotation via external deep learning APIs, and 2D structure visualization through locally implemented optimization algorithms. The chapter outlines the system requirements, describes the interface prototype, explains the technical implementation and architecture, and discusses the testing procedures employed to ensure reliability and correctness.

3.1 Requirement analysis

To provide a robust and accessible tool for computational biology, the web platform was designed according to a set of functional and non-functional requirements. Functional requirements define the services and operations that the system must provide to users, while non-functional requirements describe quality attributes such as usability, robustness, performance, and maintainability.

3.1.1 Functional requirements

- **Sequence Input Processing:** The system must accept protein sequences in standard FASTA format, capable of parsing multiple sequences simultaneously. A dedicated utility function strips FASTA headers (lines beginning with >) and validates that only legal amino acid characters are processed.
- **Function Prediction Integration:** The platform must seamlessly integrate with the external **DeepGO** API [13] to retrieve and display Gene Ontology (GO) terms categorized into three domains: Biological Process (GO:BP), Molecular Function (GO:MF), and Cellular Component (GO:CC).
- **Algorithm Selection and Configuration:** Users must be able to select from a suite of 2D structure prediction algorithms Hill Climbing (HC), Simulated Annealing (SA), Monte Carlo (MC), Replica Exchange Monte Carlo (REMC), and the trained Deep Q-Network (DQN) and configure their respective hyperparameters (e.g., number of iterations, temperature range, number of replicas, number of DQN rollouts, and visualization step interval). The final interface configuration is illustrated in Appendix A.

- **Structure Visualization:** The platform must generate a visual SVG-based representation of the folded protein on a 2D HP lattice, accurately depicting Hydrophobic (H) and Polar (P) amino acids with distinct colors, rendering the backbone chain, and reporting the total computed energy.
- **3D Structure Retrieval:** As an extended feature, the system should act as a bridge to fetch known or predicted 3D structures from the **AlphaFold DB** [17] and the **ESMFold API** [14], returning PDB-format files for further visualization.

3.1.2 Non-functional requirements

- **Responsiveness and Usability:** The interface must be responsive across different screen sizes and provide clear, user-friendly error messages for invalid inputs, API failures, or algorithm errors.
- **Robustness and Error Handling:** The system must handle malformed FASTA inputs, unavailable external APIs, invalid UniProt accessions, and algorithm execution errors without causing server crashes. In these cases, the user should receive descriptive feedback.
- **Performance Awareness:** The platform must remain suitable for interactive use by limiting the number of sequences processed per request, constraining the number of visualization frames, and using the trained DQN model for real-time inference instead of slower offline training procedures.
- **Maintainability and Modularity:** The implementation should keep the web interface, external API communication, and algorithmic logic separated. This modular structure allows new algorithms, API integrations, or interface components to be added with minimal impact on the rest of the system.

3.2 User prototype

The user interface was prototyped with a focus on simplicity, clarity, and ease of use for researchers and students unfamiliar with command-line tools. The layout is divided into the following sections:

- **Input Interface:** A prominent text area allows users to paste raw FASTA sequences. A set of radio buttons and dropdown menus positioned below the input box allow the user to select the desired action (*Predict GO Terms* or *Generate 2D Structure*) and choose a specific optimization algorithm.
- **Hyperparameter Panel:** When the *Structure* action is selected, context-sensitive input fields appear dynamically, allowing experienced users to fine-tune algorithm parameters. For example: number of iterations for HC/SA/MC, number

of replicas and temperature range for REMC, number of DQN rollouts, and the interval at which intermediate folding steps should be displayed.

- **Results Dashboard:** The output is presented in a structured panel. For function prediction, GO terms are displayed in categorized, expandable lists with their associated confidence scores. For structure prediction, the interface renders the 2D lattice graphically using an auto-scaling SVG canvas, highlighting H residues (colored) and P residues (distinct color), drawing the backbone chain, and displaying the computed minimum energy. A step explorer allows users to inspect intermediate conformations every user-defined number of iterations; for DQN, this corresponds to the constructive residue-by-residue folding trajectory. Appendix B shows representative screenshots of this step-by-step visualization from the initial partial chain to the final conformation.
- **3D Viewer Integration:** A dedicated section allows users to enter a UniProt accession code and fetch the corresponding 3D structure from AlphaFold DB or ESMFold, displaying or providing a download link to the resulting PDB file.

3.3 Implementation

The web platform was implemented using **Django 6.0.2**, a high-level Python web framework that encourages rapid development and clean, pragmatic design [15]. The overall system follows a **Model-View-Template** (**MVT**) architecture.

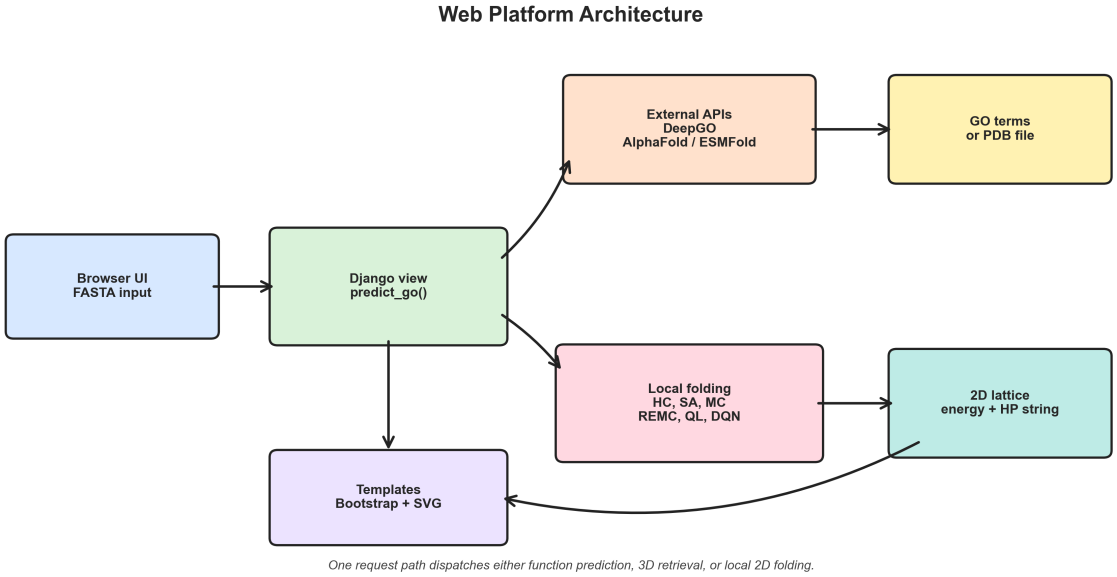

Figure 3.1: High-level architecture of the web platform. A single Django view dispatches requests to either external services for function/3D prediction or local HP-folding algorithms for 2D structure generation.

3.3.1 Backend architecture

The core logic resides in `go_predictor/views.py`, which acts as the controller:

- **Request routing:** The main view function receives `POST` requests containing the FASTA sequence, the selected action, and hyperparameter values. It dispatches processing to the appropriate function based on the action type.
- **FASTA parsing:** A utility function strips header lines and concatenates residue characters, performing basic validation (only A–Z characters representing amino acids are accepted).
- **HP translation:** The raw amino acid sequence is converted to a binary HP string using a fixed hydrophobic residue set (`ACFILMVWY`), following the standard HP model classification.

3.3.2 Algorithmic integration

The web application dynamically imports the production structure prediction scripts located in the project's `tests/` directory and the trained DQN inference module. This modular architecture decouples the web interface from the computational logic:

- `test_hc.py` — Hill Climbing (`generate_2d_structure`)
- `test_sa.py` — Simulated Annealing (`generate_2d_structure_sa`)
- `test_mc.py` — Monte Carlo (`generate_2d_structure_mc`)
- `test_remc.py` — Replica Exchange Monte Carlo (`generate_2d_structure_remc`)
- `agent_dqn_model.py` — Constructive DQN inference (`run_dqn_inference`)

Each function returns a list of (x, y) lattice coordinates and the best energy found. For interactive visualization, the classical algorithms can optionally return trace frames at a user-defined interval, while the constructive DQN returns residue-placement frames. The view passes these coordinates to the SVG renderer, which maps them to pixel positions on an auto-scaling canvas. The Tabular Q-Learning implementation remains available in `test_ql.py` for offline benchmarking, but it is not imported by the production web view because the trained DQN replaces it for real-time inference.

3.3.3 External API communication

The `requests` library is used to communicate with external services:

- **DeepGO API:** The `call_deepgo_api` function packages the user's sequence into a JSON payload and sends a `POST` request to the DeepGO inference endpoint. The returned JSON containing GO term identifiers and confidence scores for all three ontology domains is parsed and structured for display in the template.

- **AlphaFold DB:** A `GET` request is sent to the AlphaFold prediction API using the UniProt accession as query input. This retrieves the metadata associated with the specified protein. The application then extracts the `pdbUrl` field returned by the API and uses it to download or render the corresponding PDB file. This avoids hard-coding a specific AlphaFold file-version suffix such as `model_v4`.
- **ESMFold API:** A `POST` request to the ESMFold endpoint sends the raw sequence and receives a PDB-format response, allowing users to obtain predicted structures without a local installation.

All external calls are wrapped in `try-except` blocks to handle network timeouts, invalid accession codes, and API rate limits, returning descriptive error messages to the user rather than server faults.

3.3.4 Frontend design

The frontend is built with **HTML5**, **CSS3**, **Tailwind CSS**, and Lucide icons to ensure a responsive and visually consistent layout. Django's template engine is used to dynamically inject algorithmic results, loop over GO term lists, and conditionally show/hide interface sections based on the selected action. The SVG 2D structure visualization is embedded directly into the response HTML and updated client-side by JavaScript when the user moves the step slider.

3.4 Tests

Rigorous testing was conducted to ensure the platform's stability, accuracy, and robustness:

- **Integration Testing:** The Django backend was verified to correctly invoke the standalone algorithmic scripts with the parameters provided by the user interface. Edge cases were tested: sequences of length 1 (single residue), sequences composed entirely of 'P' residues (zero H-H contacts, energy $= 0$), and very long sequences (> 100 residues) to verify graceful degradation.
- **FASTA Validation:** The parser was tested against malformed inputs missing headers, sequences with non-amino-acid characters, empty submissions, and mixed-case inputs confirming that invalid data is rejected with user-friendly error messages before reaching the computational modules.
- **API Reliability:** Connections to DeepGO, AlphaFold, and ESMFold were tested under simulated failure conditions (invalid accession codes, network unavailability). Exception handling confirmed that the server returns descriptive error messages rather than `500 Internal Server Error` responses.

- **Visualization Correctness:** The SVG renderer was validated by comparing the rendered structures against manually computed expected positions for small sequences (up to 10 residues), verifying that backbone links connect consecutive residues and that H/P color assignments are consistent with the HP string.
- **Hyperparameter Passthrough:** It was verified that user-specified hyperparameter values (e.g., `iterations=50000`, temperature values, replica counts, DQN rollouts, and step-visualization intervals) are correctly parsed from the form POST data, validated to be in acceptable ranges, and passed to the underlying algorithm functions.

3.5 Summary

This chapter described the design and implementation of the web platform that serves as the user-facing component of the project. The requirement analysis guided the development of a functional interface prototype, which was implemented using Django's MVT architecture, cleanly separating web request handling from the computationally intensive algorithmic logic. The modular algorithmic integration allows the implemented algorithms (HC, SA, MC, REMC and DQN) to be invoked through a unified interface. External API communication with DeepGO, AlphaFold, and ESMFold was robustly handled with appropriate error management. Comprehensive testing confirmed that the platform correctly handles user inputs, algorithm execution, step visualization, and external API interactions across a range of normal and edge-case scenarios.

4 Algorithm development

This chapter focuses on the practical implementation of the computational methods used to solve the 2D HP protein folding problem. It describes the full prediction pipeline from raw sequence to lattice coordinates, details the specific implementation of each algorithm, and presents a quantitative comparative analysis of their performance on a benchmark dataset of 20 real UniProt protein sequences.

4.1 Protein 2D structure prediction pipeline

The structure prediction pipeline processes a raw amino acid sequence through the following stages:

1. **HP Translation:** The input sequence is scanned character by character. Each amino acid is classified as Hydrophobic (H) if it belongs to the set `{A, C, F, I, L, M, V, W, Y}` or Polar (P) otherwise, producing a binary HP string of the same length.
2. **Initialization:** The protein is initialized as a straight horizontal line: residue i is placed at position $(i, 0)$. This is always a valid self-avoiding walk (SAW) and provides a neutral, unbiased starting conformation with energy 0 (no H-H contacts in a straight line unless residues happen to be placed adjacently by the walk).
3. **Energy Evaluation:** At each step, the conformation is evaluated using the HP energy function defined in Equation 2.1. In the implementation, this is computed by building a dictionary `{(x,y): index}` from the current positions, then for each H residue iterating over its four Manhattan neighbours and counting non-consecutive H-H contacts. The result is divided by 2 (since each contact is counted twice) and negated to give a negative energy.
4. **Optimization:** The chosen algorithm iteratively modifies the conformation using the shared move set (end-flip, kink-jump, crankshaft, pivot) and its specific acceptance criterion.
5. **Output:** The algorithm returns the best conformation found (as a list of (x, y) coordinates), the corresponding minimum energy, and any learned parameters (Q-table or neural network weights).

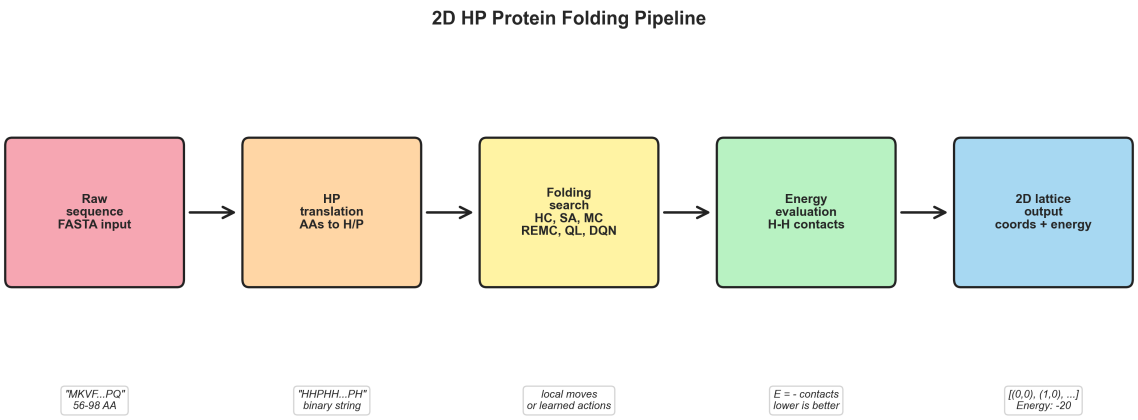

Figure 4.1: End-to-end 2D HP protein folding pipeline, from raw FASTA sequence to HP translation, optimization, energy evaluation and final lattice coordinates.

4.2 Classical heuristic algorithms

4.2.1 Hill Climbing implementation

The HC implementation (`test_hc.py`) uses the acceptance criterion $E' \leq E$ (Equation 2.2). Moves are sampled with non-uniform weights $[0.2, 0.3, 0.3, 0.2]$ for {end-flip, kink-jump, crankshaft, pivot} respectively, to avoid the high rejection rate of pivot moves in compact conformations. The algorithm maintains both the current conformation (for the random walk) and a separately tracked global best. HC was evaluated at 1,000, 10,000, 50,000, and 100,000 iterations.

4.2.2 Simulated Annealing implementation

The SA implementation (`test_sa.py`) applies the Metropolis acceptance criterion (Equations 2.3–2.4) with optimized parameters $T_0 = 30.0$ (initial temperature) and $T_f = 0.001$ (final temperature). The cooling schedule follows a geometric/exponential scheme: $T(i) = T_0 \cdot (T_f/T_0)^{i/N_{max}}$. The same weighted move sampling as HC is used. The probabilistic acceptance of worse moves is implemented as:

```
if random.random() < math.exp(-delta_e / T):
    current_positions = new_positions
```

The higher initial temperature $T_0 = 30.0$ (compared to earlier implementations at 10.0) provides more aggressive initial exploration, allowing escapes from poor local minima early in optimization. The lower final temperature $T_f = 0.001$ enables fine-grained refinement toward the end. The global best is updated only when strictly improving moves are found, while the current conformation is allowed to worsen temporarily during exploration phases.

4.2.3 Monte Carlo and REMC implementation

The MC implementation (`test_mc.py`) runs at a fixed temperature (constant throughout the run), applying the Metropolis criterion at each step without a cooling schedule. The benchmark experiments reported in this chapter use the script-level baseline temperature $T = 2.0$, while the web interface exposes this value as a configurable parameter and uses a default of $T = 5.0$ for interactive exploration. This distinction separates the controlled experimental baseline from the user-facing configuration.

The REMC implementation (`test_remc.py`) supports a configurable number of independent replicas distributed across a logarithmic temperature ladder ranging from $T_{min} = 0.1$ to $T_{max} = 30.0$, following the same multi-temperature principle used in REMC applications to HP-model protein folding [18]. The benchmark experiments use $N = 10$ replicas to keep runtime comparable across the 20-protein dataset, whereas the web platform exposes the replica count to the user and uses $N = 20$ as its interactive default. Temperature values are computed as $T_k = T_{min} \cdot (T_{max}/T_{min})^{k/(N-1)}$ for replica $k = 0, 1, \ldots, N-1$. At every 200 MC steps, adjacent replicas attempt a Metropolis-style swap with probability $P_{swap} = \min(1, \exp((\beta_k - \beta_{k+1})(E_k - E_{k+1})))$ where $\beta = 1/T$. This multi-temperature strategy enables configurations from higher-temperature (exploratory) replicas to migrate toward the colder (exploitative) replicas and vice versa, significantly improving landscape exploration while maintaining computational tractability.

4.3 Reinforcement learning implementation

4.3.1 Tabular Q-Learning

The Tabular Q-Learning agent (`test_ql.py`) is implemented using a `defaultdict` to represent the Q-table, initializing all $Q(s, a) = 0$. The state encoding (`positions_to_state`) normalizes the conformation by translating the chain so that the first residue is at the origin and rotating so that the second residue is always on the positive x-axis, thereby mapping all rotationally equivalent conformations to the same state key. Training parameters are $\alpha = 0.1$, $\gamma = 0.95$, $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.05$.

After training, a greedy exploitation pass (`exploit_q_table`) runs for an additional 500–1000 steps with $\varepsilon = 0$, using the learned Q-table to extract the best policy discovered.

4.3.2 Deep Q-Network (DQN)

The constructive DQN agent (`colab_dqn_final.py`) is implemented using the **PyTorch** framework [16]. The CNN architecture (described in detail in Section 2.6.2) takes as

input a $2 \times 21 \times 21$ float tensor and outputs a 3-dimensional Q-value vector corresponding to the relative actions Forward, Left and Right. Training uses:

- **Optimizer:** Adam, learning rate $\eta = 10^{-3}$
- **Replay buffer capacity:** 100,000 transitions
- **Mini-batch size:** 128
- **Target network update frequency:** every 1,000 optimization steps
- **Discount factor:** $\gamma = 0.99$
- **Additional stabilization:** Double DQN target selection and gradient clipping with maximum norm 10

The trained weights are saved to `dqn_weights.pth`. At inference time, the constructive policy places residues sequentially while maintaining a self-avoiding walk:

1. **State encoding:** The partial chain is encoded in a local rotation-invariant grid centered on the current chain head.
2. **Action selection:** The policy selects the relative direction for the next residue, choosing among Forward, Left and Right.
3. **Reward feedback:** New non-consecutive H-H contacts provide positive reward, collisions terminate the episode with penalty, and full-chain completion receives a small bonus.

4.4 Result analysis

4.4.1 Benchmark dataset

All algorithms were benchmarked on a dataset of 20 real protein sequences retrieved from UniProt [1], ranging from 56 to 98 amino acids in length, with H content between 20% and 53% of total residues. Sequences were tested at four iteration budgets: 1,000, 10,000, 50,000, and 100,000. Each configuration was run multiple times; best, average and standard deviation of energies are reported. Performance metrics include wall-clock execution time measured on standard CPU hardware (Intel i7, no GPU acceleration).

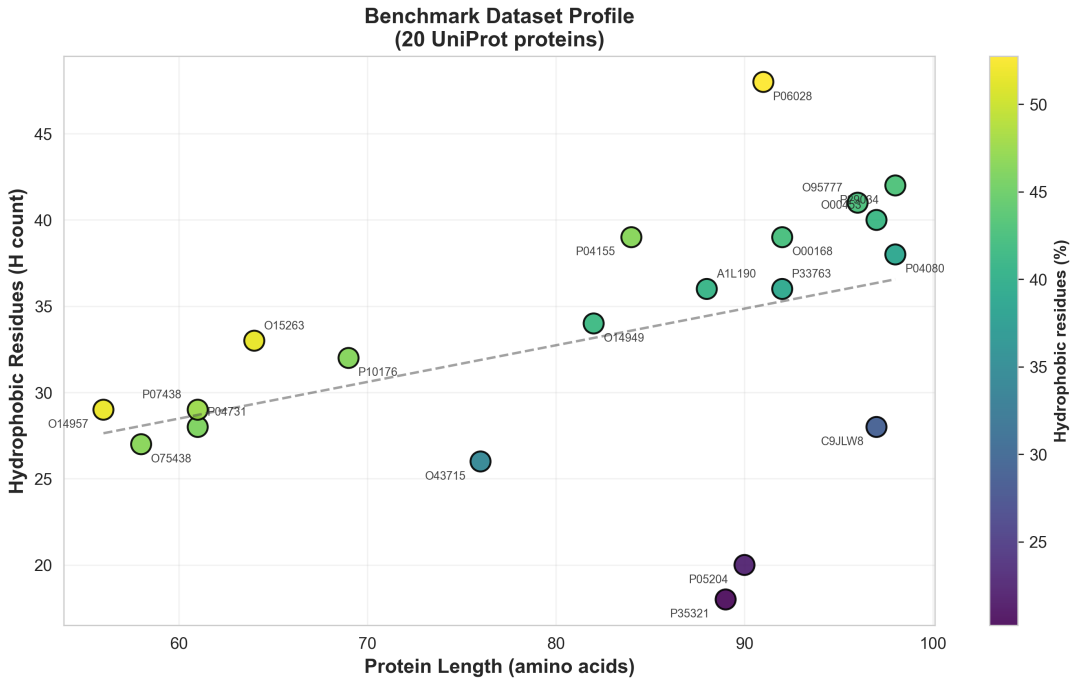

Figure 4.2: Benchmark dataset profile showing sequence length and hydrophobic content for the 20 UniProt proteins used in the experiments. The color scale represents hydrophobic residue percentage.

4.4.2 Comparison of classical heuristics (HC, SA, MC)

The complete detailed comparative analysis of all three algorithms (MC, HC, SA) is reported below for the 20 benchmark proteins at 100,000 iterations, including best energy, average energy and variance (standard deviation) from 25 independent runs each.

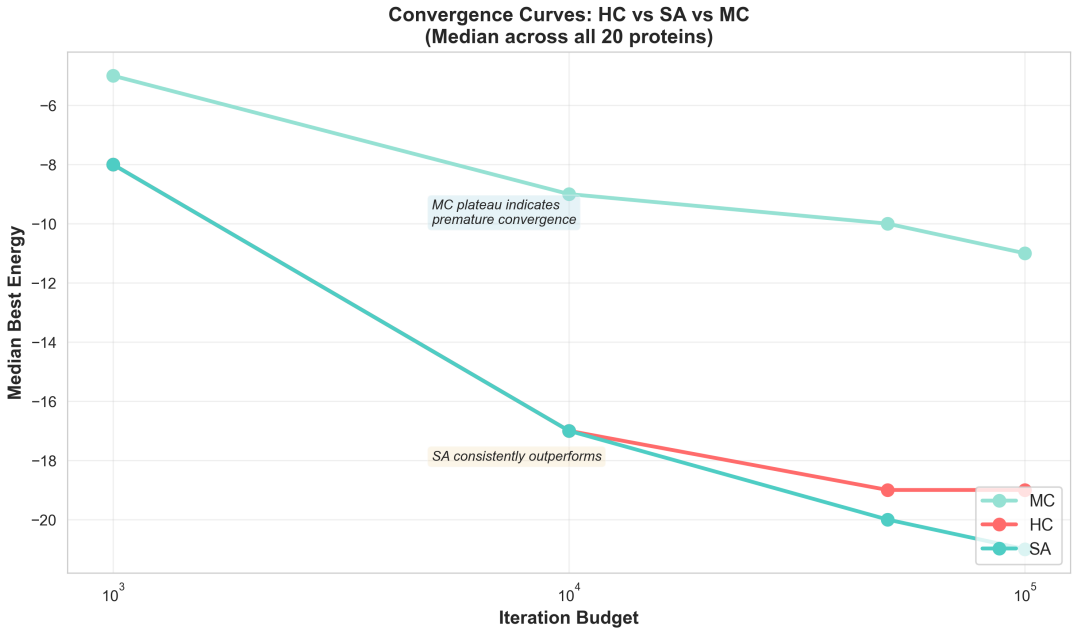

Figure 4.3: Median convergence curves for MC, HC and SA across the 20-protein benchmark. The log-scaled x-axis highlights how solution quality changes with increasing iteration budget.

Table 4.1: Detailed benchmark: MC, HC, SA at 100,000 iterations (all 20 UniProt proteins). Best energy, average and standard deviation from 25 independent runs per algorithm-protein pair.

Protein	Len	MC			HC			SA			H-cnt
		Best	Avg	Var	Best	Avg	Var	Best	Avg	Var	
A1L190	88	−13	-11.12	0.97	−24	-21.08	1.96	−25	-21.92	1.73	36
C9JLW8	97	−9	-7.56	0.82	−20	-17.36	1.8	−19	-16.8	1.44	28
O00168	92	−16	-13.36	1.08	−27	-22.4	2.35	−27	-24.12	2.03	39
O00453	97	−14	-12.48	0.87	−26	-22.04	2.39	−28	-23.28	1.93	40
O14949	82	−11	-10.4	0.58	−25	-19.4	2.31	−24	-20.52	1.94	34
O14957	56	−13	-11.24	1.01	−20	-16.64	1.82	−22	-18.48	1.81	29
O15263	64	−14	-12.08	0.86	−24	-20.24	1.71	−26	-20.64	1.91	33
O43715	76	−11	-8.36	0.76	−19	-16.16	1.84	−21	-17.04	1.17	26
O75438	58	−11	-9.04	0.73	−15	-13.44	1.71	−16	-14.36	0.99	27
O95777	96	−12	-11.04	0.84	−27	-22.32	2.63	−29	-25.52	1.81	41
P04080	98	−12	-9.64	0.81	−24	-20.28	2.26	−25	-22.04	1.86	38
P04155	84	−14	-11.88	0.88	−25	-21.56	2.31	−26	-23.4	1.63	39
P04731	61	−11	-9.4	0.71	−18	-15.04	1.77	−18	-16.12	1.39	28
P05204	90	−6	-4.96	0.61	−13	-11.48	1.0	−13	-11.2	0.82	20
P06028	91	−18	-15.72	1.24	−32	-26.64	2.55	−32	-28.36	1.6	48
P07438	61	−11	-9.8	0.82	−21	-15.96	1.97	−20	-17.28	1.57	29
P10176	69	−13	-11.64	0.76	−24	-19.44	1.61	−25	-20.88	1.74	32
P29034	98	−16	-12.92	1.15	−28	-24.24	2.33	−30	-26.44	1.58	42
P33763	92	−14	-10.56	1.04	−25	-20.68	1.93	−25	-22.68	1.52	36
P35321	89	−4	-3.04	0.2	−8	-6.76	0.93	−8	-6.4	0.96	18

Interpretation: The table is organized by increasing algorithm quality: MC (worst),

HC (intermediate), SA (best).

Monte Carlo (constant-T): Achieves the poorest performance across all 20 proteins with mean best energy -8.9. Best energies range from -4 (low-H proteins like P35321) to -18 (high-H like P06028). Variance (column "Var") is lowest overall (0.2–1.24), indicating rapid convergence to suboptimal local minima and demonstrating the critical importance of temperature control.

Hill Climbing: Intermediate performance with mean best energy -20.2, mean average -16.1. Best energies range -8 to -32 reflecting strong dependence on protein H-content and energy landscape. Variance 0.93–2.63 is higher than MC, showing greater run-to-run variability and better exploration. HC achieves competitive results on high-H proteins but stalls on low-H sequences.

Simulated Annealing (optimized): Best quality–cost profile among the single-chain classical baselines, with mean best energy -19.8 and mean average energy -16.5. With parameters $T_0 = 30.0$, $T_f = 0.001$, SA improves the average-energy profile over HC for most proteins and achieves clear best-energy gains in representative cases (e.g., O00453: SA best -28 vs HC -26). Variance ranges 0.99–2.03, providing good exploration while maintaining solution quality. SA consistently provides robust solutions across diverse protein types.

Variance Analysis: Lower variance does not automatically yield better solutions. MC's low variance (0.2–1.24) reflects premature convergence to poor local minima. SA balances variance (1.17–2.03 on average) with quality, enabling effective landscape exploration. This demonstrates that controlled variability is essential for escaping suboptimal attractors.

Recommendation: Simulated Annealing is the optimal choice for routine production use because it offers strong solution quality with well-balanced variance and low runtime. REMC remains preferable when the objective is absolute minimum energy and the higher computational cost is acceptable.

4.4.3 Replica Exchange Monte Carlo (REMC) results

The REMC benchmark is reported below using 10 replicas distributed on a logarithmic temperature ladder ($T_{min} = 0.1$ to $T_{max} = 30.0$), with swap attempts every 200 MC steps.

Table 4.2: REMC (10 replicas, logarithmic temperature ladder) at 100,000 iterations: best, average and standard deviation from multiple runs across all 20 UniProt proteins.

Protein	**Len**	**Best**	**Avg**	**Std**	**Time (s)**
A1L190	88	-25	-24.33	0.58	23.34
C9JLW8	97	-20	-19.0	1.0	21.95
O00168	92	-27	-26.67	0.58	24.77
O00453	97	-26	-25.0	1.0	24.84
O14949	82	-23	-23.0	0.0	22.49
O14957	56	-22	-21.33	0.58	16.91
O15263	64	-26	-24.0	1.73	19.1
O43715	76	-20	-19.33	0.58	19.11
O75438	58	-17	-16.33	0.58	16.45
O95777	96	-27	-26.67	0.58	24.94
P04080	98	-26	-24.67	1.15	23.56
P04155	84	-28	-27.0	1.0	23.61
P04731	61	-18	-18.0	0.0	17.28
P05204	90	-13	-12.67	0.58	18.68
P06028	91	-32	-32.0	0.0	25.47
P07438	61	-21	-20.33	0.58	17.54
P10176	69	-25	-23.67	1.15	20.18
P29034	98	-30	-28.67	1.53	25.58
P33763	92	-26	-25.33	0.58	24.17
P35321	89	-8	-7.67	0.58	18.17
Median	87	-24	-23.0	1.2	21.95
Mean	82	-23.6	-22.6	1.5	20.77

Analysis: REMC achieves median best energy -24 (vs SA -20), a consistent 4-unit improvement via parallel tempering. Average energies range -7.67 to -32.0 with significantly lower standard deviations (0.0–1.73) than HC/SA, indicating high run-to-run consistency. Wall-clock execution times are 16–25 seconds, representing 20–24$\times$ computational cost compared to SA. REMC is particularly effective on high-H proteins (P06028: REMC best -32, avg -32.0, std 0.0 represents perfect consistency). REMC is recommended for applications requiring maximum solution quality when computational budget permits.

4.4.4 Tabular Q-Learning convergence and hyperparameter analysis

The Q-Learning benchmark below shows energy improvement as a function of training steps across a representative 16-protein subset of the full 20-protein benchmark. This subset was used for the tabular experiments because the Q-table grows exponentially with sequence length and repeated 100k-step runs over all 20 sequences were substantially less practical than for the classical heuristics. The DQN experiments later in this chapter return to the complete 20-protein dataset.

Table 4.3: Tabular Q-Learning convergence: best energy found at 1k, 10k, 50k, 100k steps. Average and standard deviation from multiple runs at 100k iteration budget.

Protein	1k Best	10k Best	50k Best	100k Best	100k Avg	100k Std
A1L190	-2	-6	-8	-12	-8.67	3.06
C9JLW8	-2	-3	-4	-7	-5.67	1.53
O00168	-7	-8	-10	-11	-10.33	0.58
O00453	-4	-8	-10	-9	-8.67	0.58
O14949	-5	-5	-9	-9	-8.67	0.58
O14957	-7	-7	-10	-8	-8.0	0.0
O15263	-4	-8	-10	-9	-8.33	0.58
O43715	-1	-4	-6	-7	-5.67	1.53
O75438	-5	-6	-7	-7	-6.33	0.58
O95777	-7	-5	-8	-10	-7.67	2.52
P04080	-3	-4	-5	-7	-6.0	1.0
P04155	-1	-4	-9	-10	-9.0	1.0
P04731	-4	-5	-6	-8	-6.33	1.53
P05204	-1	-2	-2	-4	-3.33	0.58
P06028	-4	-5	-13	-13	-10.33	2.31
P07438	-5	-5	-7	-8	-7.0	1.0

Q-Learning Insights: Median best energy improves from -5 at 1k steps to -9 at 100k steps, demonstrating clear learning progression (80% improvement). However, absolute gap persists: QL median -9 vs SA median -20 represents 55% underperformance at 100k steps. Standard deviations at 100k range 0.0–3.06, showing high run-to-run variability (vs REMC's 0.0–1.73), indicating that tabular QL struggles with exploration-exploitation trade-off in high-dimensional space.

Hyperparameter analysis (9 configurations tested) reveals: (1) slow ε-decay (80% exploration phase) consistently outperforms fast decay, (2) mid-range learning rates ($\alpha = 0.1$) balance convergence and stability better than aggressive learning ($\alpha = 0.9$), (3) discount factors $\gamma \in [0.9, 0.99]$ yield superior results to $\gamma = 0.5$. Best configuration: $\alpha = 0.1$, $\gamma = 0.9$, slow decay yields -10 median on A1L190.

The fundamental limitation remains the **curse of dimensionality**: for 56–98 AA proteins, the state space grows exponentially, rendering exhaustive state visitation infeasible even at 100k steps. Tabular Q-Learning remains practical only for short proteins (≤ 25 AA).

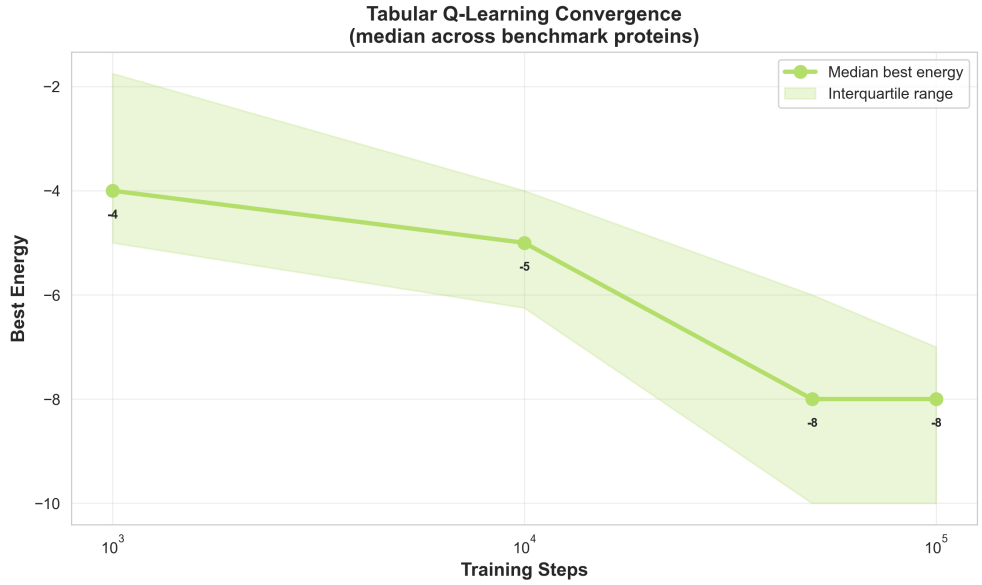

Figure 4.4: Tabular Q-Learning convergence over increasing training budgets. The line shows median best energy across benchmark proteins and the shaded band shows the interquartile range.

4.4.5 DQN results

The Constructive DQN (`colab_dqn_final.py`) was trained on the complete dataset of 20 UniProt proteins using GPU acceleration (CUDA), processing 20,000 training episodes over the entire benchmark set. Training required 23,279.627 seconds (≈ 388.0 minutes, or 6.47 hours) on GPU hardware. The constructive approach builds protein chains residue-by-residue using relative directional actions (Forward, Left, Right), rather than perturbing an already complete chain. This formulation makes each episode a sequential folding trajectory and allows the model to learn local construction policies from partial conformations.

Table 4.4: DQN training configuration used in the final Colab run.

Parameter	**Value**
Training proteins	20
Protein length range	56–98 AA
Episodes	20,000
Mini-batch size	128
Replay buffer capacity	100,000 transitions
Learning rate	0.001
Discount factor γ	0.99
ε schedule	$1.0 \rightarrow 0.05$, decay 0.995
Target-network update	every 1,000 optimization steps
Total optimization steps	1,195,355
Training time	23,279.627 s ($\approx$ 6.47 h)

Training performance summary: During training across all 20 proteins, the DQN achieved:

- **Median best energy:** -18 (across 20 proteins)
- **Mean best energy:** -16.9
- **Best achieved energy:** -26 (P06028, 91 AA, 48 H residues)
- **Energy range:** -26 to -3 (showing strong protein-dependence)
- **Mean final episode energy:** -7.76
- **Mean reward:** 7.75 per episode; mean best reward 17.62
- **Mean completion rate:** 0.57 during training
- **Greedy inference:** final trained policy completed 20/20 proteins without exploration, with median greedy energy -12

Table 4.5: Constructive DQN detailed training and greedy-inference results over the 20 UniProt benchmark proteins. Best, average and last values are computed from the 20,000-episode training run; greedy values are obtained by evaluating the final trained policy without exploration.

Protein	Len	H	Episodes	Best E	Avg E	Last E	Comp.	Greedy E
O43715	76	26	956	-14	-6.22	-6	0.61	-8
C9JLW8	97	28	985	-11	-4.25	-1	0.56	-5
O00168	92	39	982	-25	-12.18	-4	0.49	-13
P35321	89	18	992	-3	-0.68	0	0.61	-1
O14957	56	29	1037	-19	-9.84	-4	0.61	-14
P04080	98	38	975	-14	-6.17	-6	0.51	-7
P06028	91	48	968	-26	-12.51	-17	0.49	-13
O14949	82	34	984	-19	-7.09	-12	0.58	-12
P29034	98	42	1012	-22	-10.04	-9	0.48	-13
P33763	92	36	1066	-17	-7.09	-2	0.54	-13
O95777	96	41	996	-19	-8.42	-2	0.49	-13
O00453	97	40	994	-23	-12.38	-2	0.50	-12
P07438	61	29	1039	-14	-5.73	-5	0.63	-10
O15263	64	33	943	-22	-11.86	-12	0.60	-12
P05204	90	20	1036	-7	-2.32	-3	0.65	-5
A1L190	88	36	1012	-16	-6.25	-7	0.55	-7
P10176	69	32	956	-19	-8.85	-8	0.58	-11
P04731	61	28	1050	-12	-5.18	-4	0.67	-6
O75438	58	27	984	-16	-7.28	-8	0.63	-12
P04155	84	39	1033	-20	-10.94	-6	0.53	-12
Mean	82.0	33.2	1000	-16.9	-7.76	–	0.57	-10.0
Median	88.5	33.5	993	-18.0	-7.19	–	0.57	-12.0

Sequence-level interpretation: The best results occur on proteins with higher hydrophobic content: P06028 (-26, 48 H), O00168 (-25, 39 H), O00453 (-23, 40 H), P29034 (-22, 42 H), and O15263 (-22, 33 H). In contrast, low-H sequences remain difficult: P35321 contains only 18 H residues and reaches best energy -3, while P05204 contains 20 H residues and reaches best energy -7. This pattern confirms that the sparse reward structure is strongly tied to the availability of H-H contacts; when few hydrophobic residues exist, the agent receives fewer positive learning signals.

Comparison with classical methods: The DQN median best energy is -18, compared with approximately -20 for SA and -24 for REMC. Therefore, the trained DQN approaches the simulated annealing baseline but still does not outperform the best stochastic optimization method. Its value is methodological: it learns a reusable construction policy from experience, rather than re-optimizing each sequence from scratch. The remaining gap to REMC suggests that longer training, sequence-specific rollouts, policy search over Q-values, and more recent RL algorithms may be required for fully competitive performance.

Implications: The constructive environment with relative directional actions and rotation-invariant state representation successfully enables the DQN to learn non-trivial folding strategies across diverse proteins. The final deterministic policy completed all 20 chains and reached median greedy energy -12, while the best training trajectories reached median energy -18. This indicates that the learned policy is usable for web inference, but that repeated rollouts remain important for extracting stronger conformations from the same Q-network.

DQN Learning Curve

Figure 4.5 presents the DQN training curve across all 20 proteins over 20,000 episodes, showing the final energy obtained in each episode and its moving-average trend.

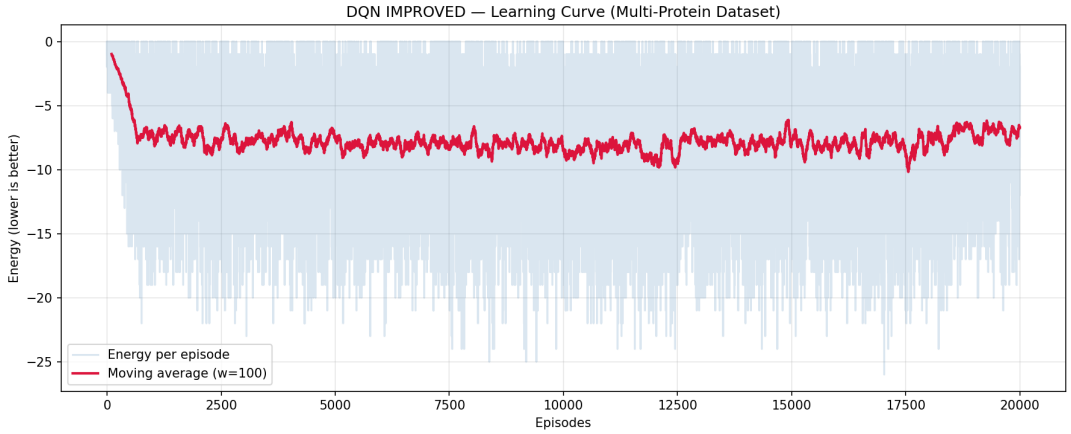

Figure 4.5: DQN training learning curve over 20,000 episodes. The raw curve reports final episode energy on mixed protein lengths, while the moving average highlights the overall learning trend. High variance reflects both stochastic exploration and H-content diversity (18–48 hydrophobic residues): high-H proteins can achieve best energies from -22 to -26, while low-H sequences remain closer to -3 to -7.

Learning Dynamics: The curve shows three broad phases: (1) *early exploration*, where ε decays rapidly from 1.0 toward 0.05 and the replay buffer begins to collect diverse partial-chain transitions; (2) *policy refinement*, where the moving-average energy stabilizes as the buffer reaches a more representative mixture of successful and failed constructions; and (3) *late low-exploration training*, where $\varepsilon = 0.05$ maintains limited exploration while the policy continues to discover stronger conformations on favorable sequences. The scatter across proteins demonstrates that even with shared network weights, sequence-specific characteristics (H-content, length, local motifs) dominate final performance, motivating the need for per-sequence fine-tuning or domain adaptation in future work.

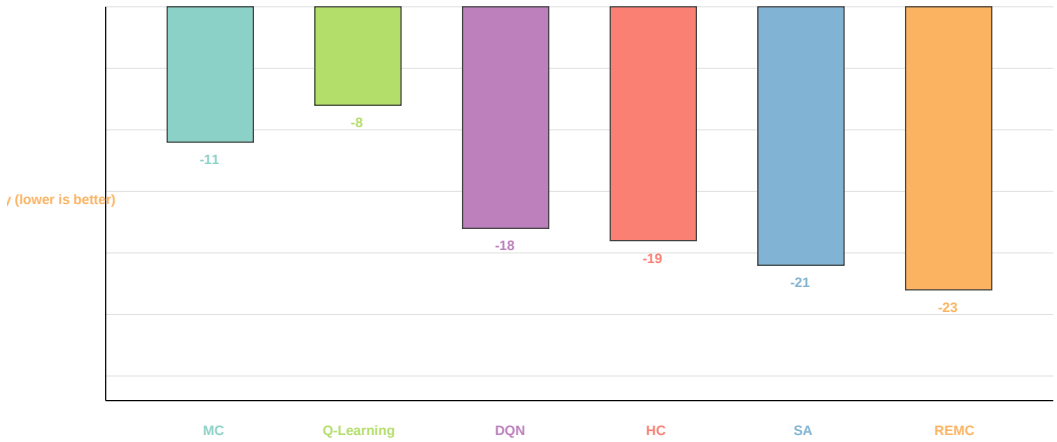

Figure 4.6: Median best energy obtained by each algorithm at the 100,000-step budget where applicable. DQN values correspond to the final constructive DQN per-sequence training results.

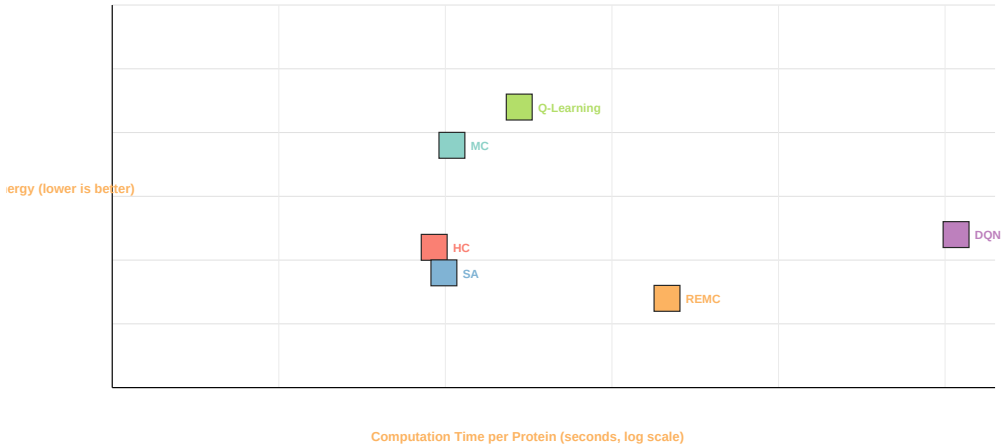

Figure 4.7: Trade-off between solution quality and computational cost. Classical methods are CPU timings at 100,000 iterations, while the DQN point summarizes training cost amortized over the 20 benchmark proteins.

4.5 Summary and performance hierarchy

This chapter presented the complete 2D HP protein folding pipeline and detailed implementation of all six algorithms. Comprehensive benchmark on 20 UniProt sequences (56–98 AA, 20–53% hydrophobicity) with detailed measurements of best energy, average energy, and variance (standard deviation) establishes the following practical performance hierarchy:

1. **Replica Exchange MC (REMC):** Achieves the **highest solution quality** (median best -24, mean -23.6) via parallel tempering across 10 temperature replicas in the benchmark setting. **Lowest variance** (0.0–1.73) indicates exceptional run-to-run consistency. Computational cost is high: 20–24$\times$ slower than SA (16–25 seconds). Appropriate when solution quality is paramount and compute budget permits.
2. **Simulated Annealing (SA): Recommended production baseline** with optimized parameters $T_0 = 30.0$, $T_f = 0.001$. Although REMC reaches better absolute energies, SA provides the best quality–cost trade-off for routine use: mean best energy -19.8, mean average -16.5, execution time <1.2s per protein, and robust behavior across all 20 proteins (H-content 20–53%). Variance 0.99–2.03 provides a good balance between exploration and exploitation, and SA improves the average-energy profile over HC for most proteins.
3. **Hill Climbing (HC):** Intermediate performance with mean best energy -20.2, mean average -16.1. Higher variance (1.71–2.63) enables better exploration than MC but shows strong landscape dependence. Best for high-H proteins; stalls on low-H sequences. Fast execution (<1.0s) but requires multiple restarts for robustness.
4. **Monte Carlo (constant-T):** Poor baseline with mean best energy -8.9 (56% degradation vs SA). **Lowest variance** (0.2–1.24) indicates rapid convergence to suboptimal local minima, demonstrating that low variance without good solutions reflects premature convergence. Conclusively demonstrates critical importance of temperature control. Not recommended for production use.
5. **Tabular Q-Learning:** Severely limited by curse of dimensionality for 56–98 AA proteins. Achieves median -9 at 100k steps (55% worse than SA). Higher variance (0.0–3.06) shows exploration difficulties. Practical only for short sequences (≤ 25 AA) where state space is manageable.
6. **Deep Q-Network (CNN-based, Constructive):** Achieves median best energy -18 and mean best energy -16.9 with GPU training over 20,000 episodes on the full 20-protein dataset. Peak performance of -26 on P06028 (91 AA, 48 H) demonstrates that the learned construction policy can discover non-trivial folds on favorable sequences; however, gaps to SA (median approximately -20) and

REMC (median approximately -24) remain. The constructive environment with a rotation-invariant 21×21 grid and relative directional actions validates DRL feasibility. Computational cost is high: 23,279.627 seconds of GPU training. Limitations stem from sparse reward structure on low-H proteins and sample inefficiency of neural Q-learning. Policy-gradient methods, search over DQN action values, and better sequence-specific rollout strategies are expected to narrow the remaining performance gap.

The progressive algorithm complexity from greedy search to learned spatial policies reflects the methodological evolution toward more principled and asymptotically scalable approaches. Future work will focus on GPU-accelerated DQN training, policy-gradient RL methods with better sample efficiency, and extension to 3D HP protein folding.

5 Conclusion

This final chapter summarizes the primary outcomes and contributions of the project, reflects on the challenges encountered during development, and outlines potential directions for future research and system enhancement.

5.1 Final remarks

This project successfully developed a comprehensive computational framework for protein analysis, integrating both structure prediction and functional annotation within a single user-accessible web platform. The system bridges the gap between raw biological sequences and complex structural and functional predictions, offering researchers and students an accessible tool that requires no local installation of heavy computational infrastructure.

5.1.1 Web platform

The Django-based web platform was successfully designed and deployed, providing a clean, responsive interface that supports multiple 2D structure prediction algorithms alongside external deep learning API integration. The modular architecture decoupling the web request handler from the algorithm scripts proved effective: algorithms could be updated, replaced, or extended without modifying the web server code. The FASTA parser, hyperparameter controls, SVG-based structure visualization, step-by-step folding explorer, and error handling were all validated through systematic testing.

5.1.2 Classical optimization

A progressive suite of classical heuristic algorithms was implemented and rigorously benchmarked across 20 real UniProt protein sequences of 56–98 amino acids:

- **Hill Climbing** proved fast and competitive at high iteration counts (achieving best energies of -25 to -30 at 100,000 iterations for H-rich sequences) but consistently stalled in local minima at lower budgets.
- **Simulated Annealing** demonstrated a clear advantage over HC on average across the dataset (median improvement of ≈ 1.5 energy units at 100,000 iterations), confirming that the Metropolis acceptance criterion and the exponential cooling schedule effectively prevent entrapment in local minima.

- **Monte Carlo** (constant temperature) underperformed both HC and SA, validating the importance of adaptive temperature control for effective conformation space exploration.
- **Replica Exchange Monte Carlo** achieved the strongest overall solution quality, with median best energy around -24, but at approximately 20–24$\times$ the computational cost of SA, making it most suitable when solution quality is paramount and runtime is not a constraint.

5.1.3 Reinforcement learning

The introduction of Reinforcement Learning represented the most significant methodological advance of the project:

- **Tabular Q-Learning** successfully learned folding policies for short sequences, demonstrating convergence and the ability to discover non-trivial conformations through ε-greedy exploration and Bellman updates. For sequences of up to ≈ 25 residues, it provides a principled alternative to stochastic heuristics with theoretical convergence guarantees. For longer sequences (56–98 AA), it is outperformed by classical methods due to the exponential growth of the state space.
- **Deep Q-Network (DQN)** (constructive implementation, `colab_dqn_final.py`): implemented in PyTorch using a constructive, relative-direction formulation. Rather than perturbing an existing straight-line conformation, the agent **builds the protein one residue at a time**, choosing relative directions (Forward, Left, Right) to place the next amino acid. The environment enforces self-avoiding walks: collisions terminate an episode with a strong penalty.

  The state is represented as a rotation-invariant $2 \times 21 \times 21$ grid centered on the chain head and rotated so the current direction always points upward; channel 0 encodes Hydrophobic residues and channel 1 encodes Polar residues. This compact, local representation drastically reduces the effective state space and improves generalization across sequences of different lengths and H-content.

  Training configuration used in the GPU-accelerated Colab implementation: dataset of 20 UniProt sequences (56–98 AA), 20,000 episodes, replay buffer capacity 100,000, mini-batch size 128, Adam optimizer with $\eta = 10^{-3}$, $\gamma = 0.99$, ε decayed from 1.0 to 0.05, target-network updates every 1,000 optimization steps, Double DQN target selection, gradient clipping, and reward shaping for low-H-density proteins. Trained on CUDA devices with total training time 23,279.627 seconds (≈ 6.47 hours). Trained weights are saved to `dqn_weights.pth`.

  Quantitative performance: median best energy across 20 proteins of -18, mean best energy -16.9, and range -26 (P06028) to -3 (P35321). Mean final episode energy was -7.76, mean reward was 7.75, and the mean training completion rate

was 0.57. Greedy inference completed all 20 proteins with median greedy energy -12. While performance still trails REMC and remains slightly below the SA baseline, the constructive DQN demonstrates that learned policies can discover non-trivial conformations through reinforcement learning on local lattice representations. The approach validates the viability of DRL for this domain and establishes a foundation for policy-gradient methods, multi-rollout inference, and enhanced reward design.

Overall, the project validates the progressive hypothesis: stochastic methods with temperature control outperform greedy search; learned policies scale beyond the limitations of tabular methods; and spatial CNN representations provide a viable foundation for deep reinforcement learning on lattice protein models.

5.2 Future works

Several promising directions exist to extend and refine this work:

- **DQN Training at Scale:** The most immediate next step is to train the DQN for a larger number of episodes (e.g., 10^5–10^6) with GPU acceleration. Hyperparameter tuning (learning rate, batch size, replay buffer capacity, target network update frequency) using systematic search methods (e.g., Bayesian optimization) could substantially increase solution quality.
- **Alternative DRL Architectures:** Beyond DQN, more recent algorithms such as Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) could be explored. These policy-gradient methods offer better sample efficiency and stability for continuous-action environments, which may be beneficial for extended move sets.
- **3D HP Model Extension:** Currently, the DRL agent operates within the constraints of the 2D HP lattice. A natural and scientifically significant progression is to extend the environment to a 3D cubic lattice (or a face-centered cubic lattice), incorporating a more realistic energy function with side-chain interactions, moving closer to state-of-the-art structure prediction.
- **Database Integration and Analytics:** Persistently storing user-submitted sequences, their GO predictions, and their predicted structures in a relational database would enable longitudinal analysis, user history tracking, and the creation of a searchable internal dataset for research use.
- **Validation with Experimental Data:** The structural predictions produced by all algorithms should be systematically compared against experimentally determined structures from the Protein Data Bank (PDB) using standard metrics such as the Template Modelling score (TM-score) or root mean square deviation (RMSD),

to rigorously quantify real-world accuracy.

- **Unified Structure-Function Model:** An ambitious future direction is to develop an end-to-end architecture where structural embeddings generated by the DQN (or a graph neural network operating on the lattice) are directly used as inputs to a local function prediction head, deeply integrating the structure-to-function relationship within a single, unified deep learning pipeline.

References

[1] The UniProt Consortium, "UniProt: The universal protein knowledgebase in 2023," *Nucleic Acids Research*, vol. 51, no. D1, pp. D523–D531, 2023.

[2] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, and P. E. Bourne, "The protein data bank," *Nucleic Acids Research*, vol. 28, no. 1, pp. 235–242, 2000.

[3] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Zidek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis, "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[4] K. A. Dill, "Theory for the folding and stability of globular proteins," *Biochemistry*, vol. 24, no. 6, pp. 1501–1509, 1985.

[5] P. Crescenzi, D. Goldman, C. Papadimitriou, A. Piccolboni, and M. Yannakakis, "On the complexity of protein folding," *Journal of Computational Biology*, vol. 5, no. 3, pp. 423–465, 1998.

[6] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[8] K. Hukushima and K. Nemoto, "Exchange monte carlo method and application to spin glass simulations," *Journal of the Physical Society of Japan*, vol. 65, no. 6, pp. 1604–1608, 1996.

[9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: https://mitpress.mit.edu/9780262039246/reinforcement-learning/

[10] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[11] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[12] M. Ashburner, C. A. Ball, J. A. Blake, D. Botstein, H. Butler, J. M. Cherry, A. P. Davis, K. Dolinski, S. S. Dwight, J. T. Eppig, M. A. Harris, D. P. Hill, L. Issel-Tarver, A. Kasarskis, S. Lewis, J. C. Matese, J. E. Richardson, M. Ringwald, G. M. Rubin, and G. Sherlock, "Gene ontology: Tool for the unification of biology," *Nature Genetics*, vol. 25, no. 1, pp. 25–29, 2000.

[13] M. Kulmanov, M. A. Khan, and R. Hoehndorf, "DeepGO: Predicting protein functions from sequence and interactions using a deep ontology-aware classifier," *Bioinformatics*, vol. 34, no. 4, pp. 660–668, 2018.

[14] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli, Y. Shmueli, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives, "Evolutionary-scale prediction of atomic-level protein structure with a language model," *Science*, vol. 379, no. 6637, pp. 1123–1130, 2023.

[15] Django Software Foundation, *Django Documentation*, 2024, accessed: 2026-05-19. [Online]. Available: https://docs.djangoproject.com/en/4.2/

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

[17] M. Varadi, S. Anyango, M. Deshpande, S. Nair, C. Natassia, G. Yordanova, D. Yuan *et al.*, "AlphaFold protein structure database: Massively expanding the structural coverage of protein-sequence space with high-accuracy models," *Nucleic Acids Research*, vol. 50, no. D1, pp. D439–D444, 2022.

[18] C. Thachuk, A. Shmygelska, and H. H. Hoos, "A replica exchange monte carlo algorithm for protein folding in the HP model," *BMC Bioinformatics*, vol. 8, no. 342, 2007.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[20] OpenAI, "Chatgpt," https://chat.openai.com/, 2026, large language model used

as an auxiliary tool for code implementation, debugging assistance, and general guidance during the research process.

Appendices

Anexo A - Web Platform Input and Configuration

This appendix documents the main input and configuration screens of the developed web platform. The screenshots show the user workflow from FASTA sequence submission to algorithm, hyperparameter, and step-visualization configuration.

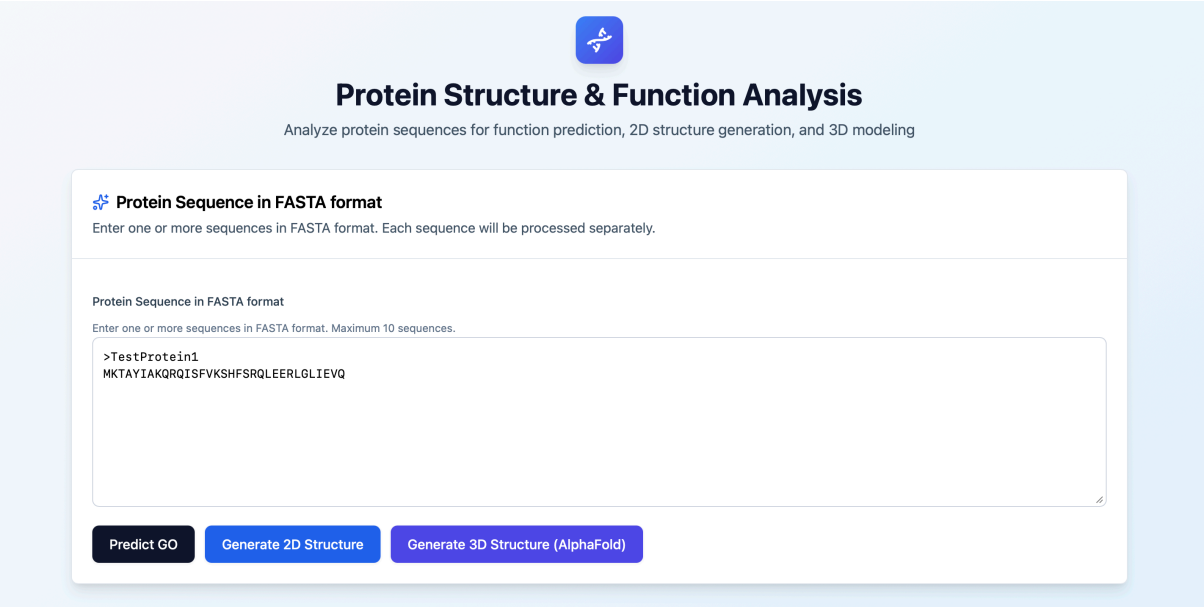

Figure 5.1: FASTA input screen of the web platform. Users can submit one or more protein sequences and choose between GO prediction, 2D HP folding, and 3D structure retrieval.

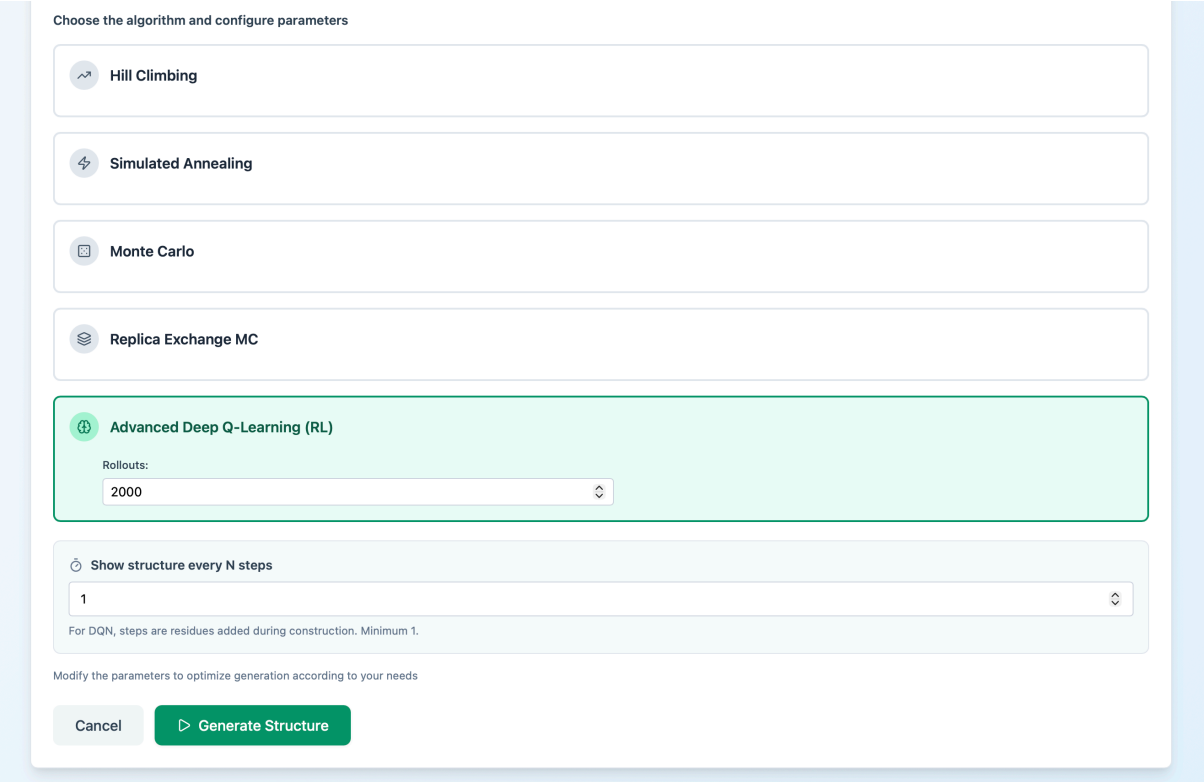

Figure 5.2: Algorithm selection screen for 2D structure generation. The interface exposes the implemented optimization and reinforcement learning methods through a unified workflow. The selected DQN configuration shows the rollout parameter and the step-visualization interval, allowing users to choose how frequently intermediate conformations should be recorded and displayed.

Figure 5.3: Parameter configuration panel for 2D HP structure generation. Algorithm-specific controls allow users to tune iteration budgets, temperatures, replicas, learning parameters, and the interval used by the step explorer to display intermediate folding states.

Anexo B - Web Platform Results

This appendix presents representative output screens produced by the web platform, including Gene Ontology prediction, 2D HP lattice visualization with step-by-step exploration, and interactive 3D structure retrieval.

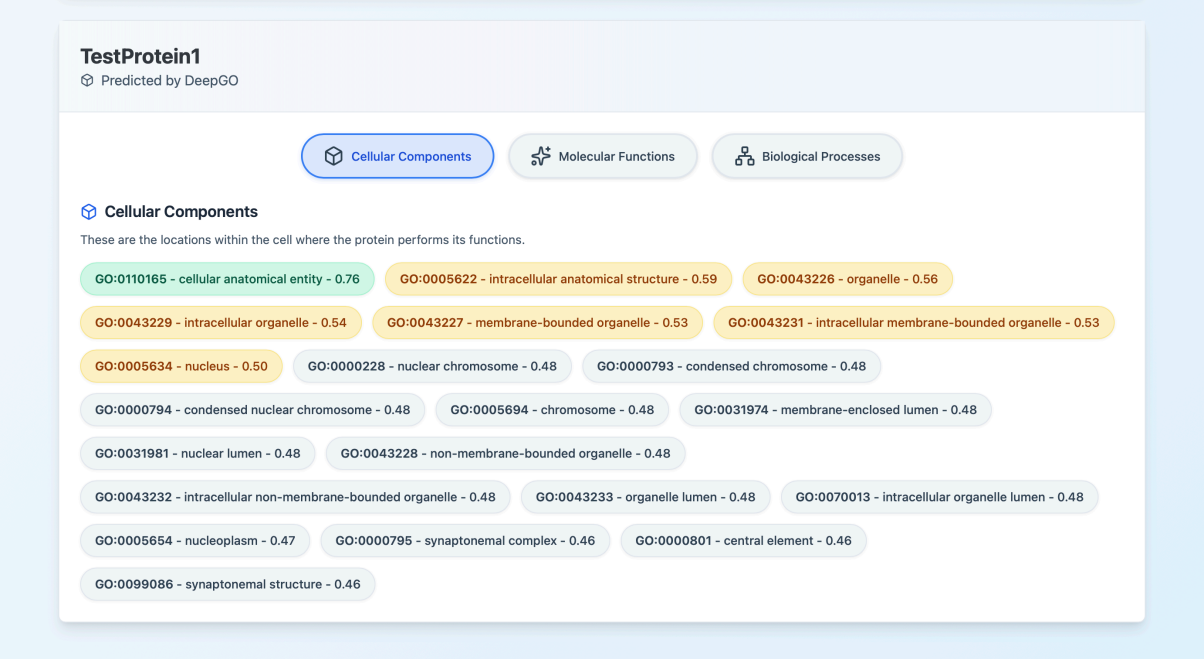

Figure 5.4: GO prediction results screen. Predicted Gene Ontology terms are grouped by biological process, molecular function, and cellular component when returned by the external DeepGO service.

Figures 5.5–5.9 illustrate the step explorer added to the 2D structure visualization workflow. Instead of showing only the final conformation, the platform stores intermediate frames according to the user-defined interval and lets the user navigate the folding trajectory with previous, play, next, and slider controls. For the constructive DQN, each displayed step corresponds to one or more residues added during chain construction; for classical algorithms, each displayed step corresponds to a sampled optimization interval.

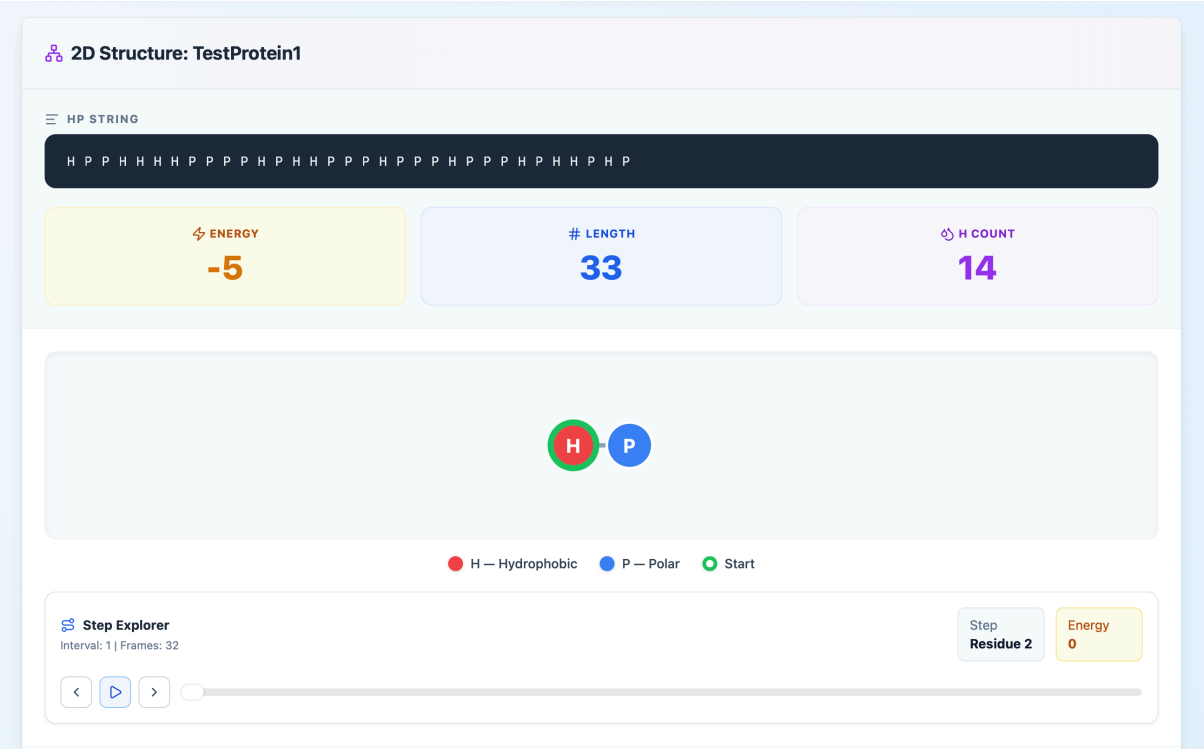

Figure 5.5: Initial 2D HP step-explorer frame. The protein construction starts from the first residues, with the current step, energy, frame count, and playback controls displayed below the SVG lattice.

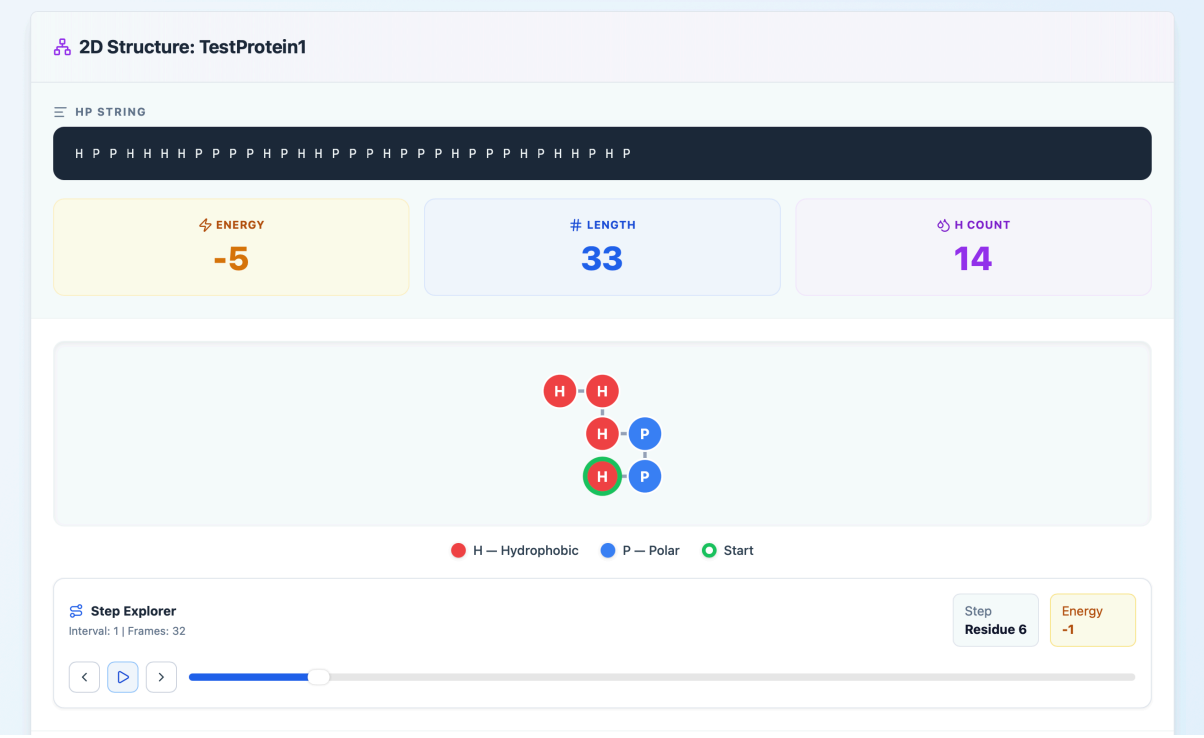

Figure 5.6: Early folding frame in the 2D HP step explorer. The interface shows how the partial chain grows while preserving the same energy, sequence, and navigation context.

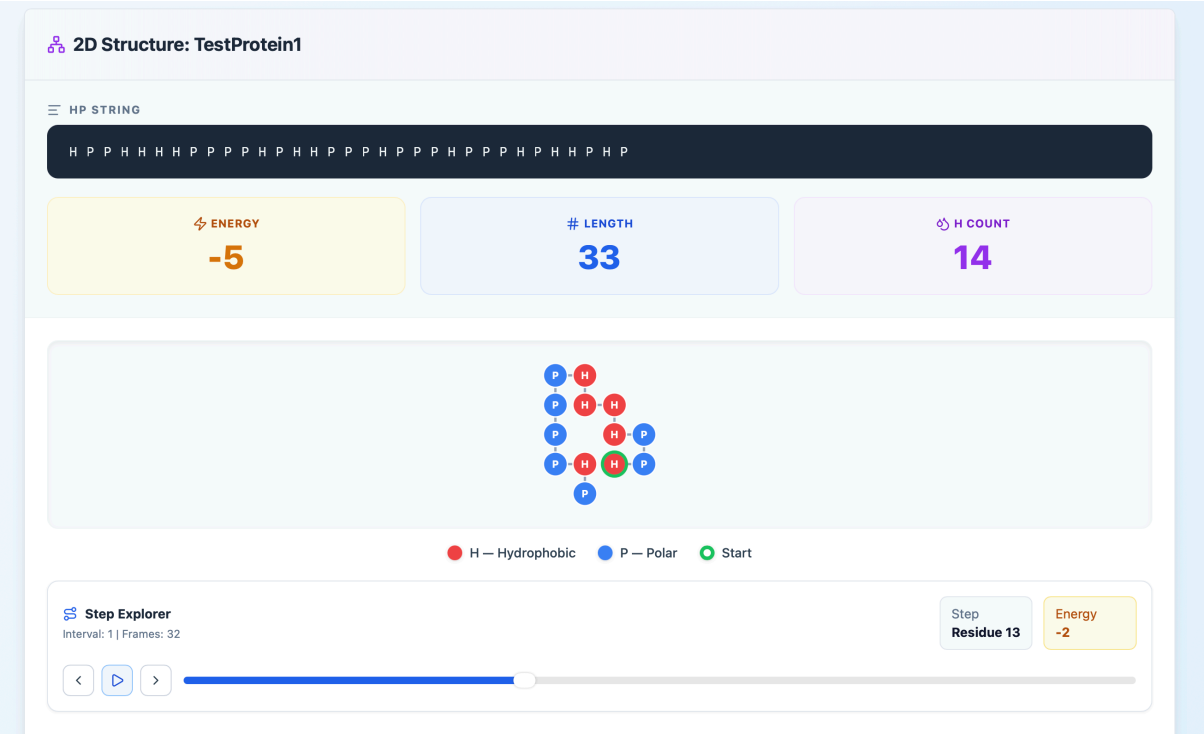

Figure 5.7: Intermediate folding frame in the 2D HP step explorer. The visualization makes it possible to inspect how local turns and hydrophobic contacts emerge before the final conformation is reached.

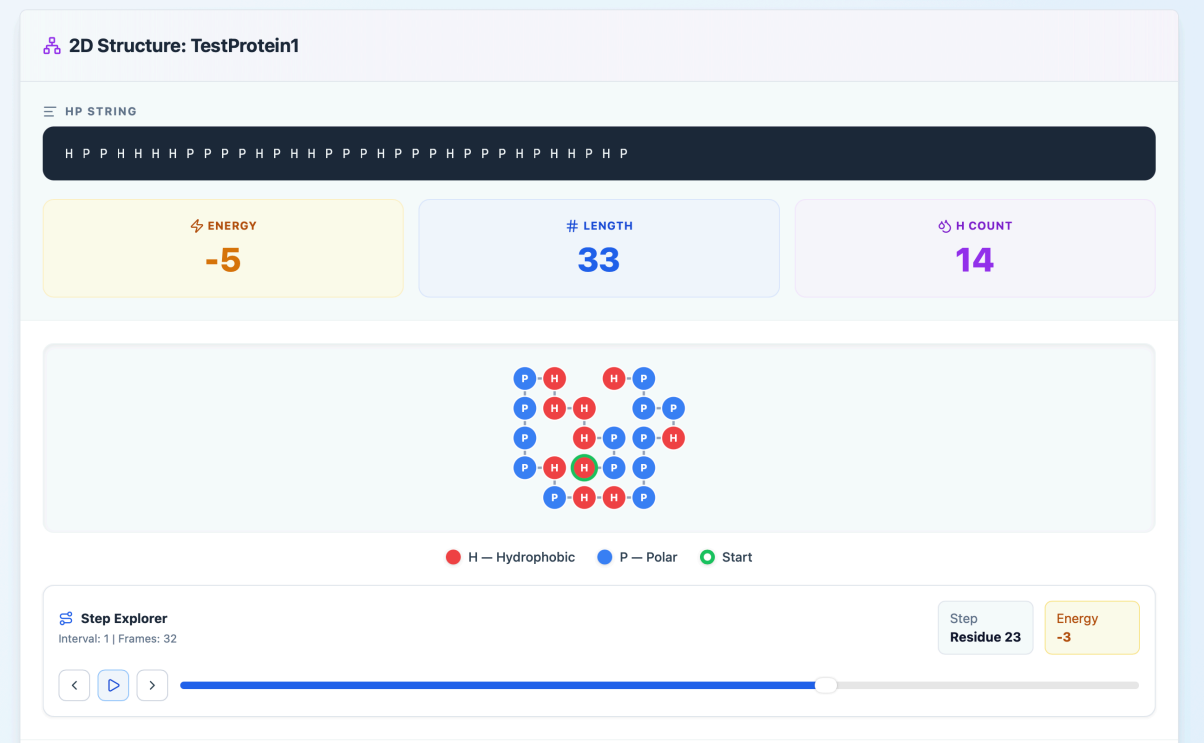

Figure 5.8: Late folding frame in the 2D HP step explorer. The slider position and step counter allow the user to compare this near-final state with earlier conformations.

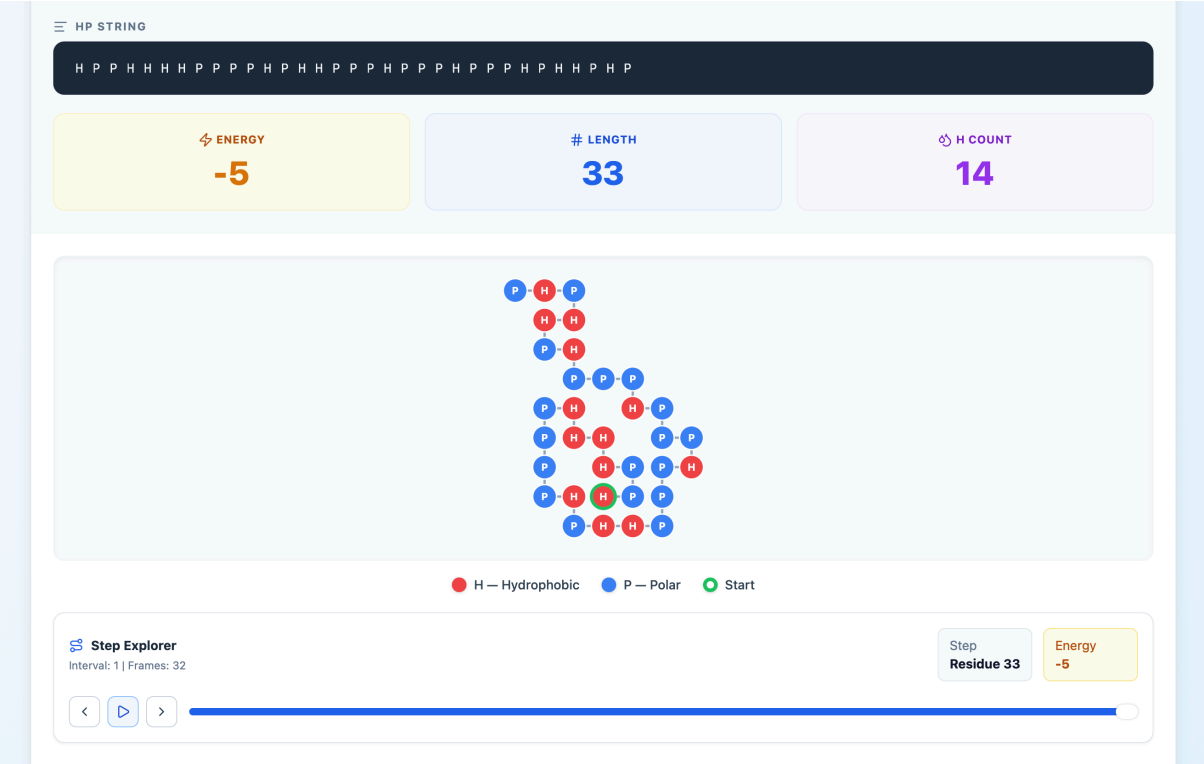

Figure 5.9: Final 2D HP structure generated by the platform. The completed lattice conformation is shown together with the HP string, final energy, sequence length, hydrophobic residue count, and final step-explorer position.

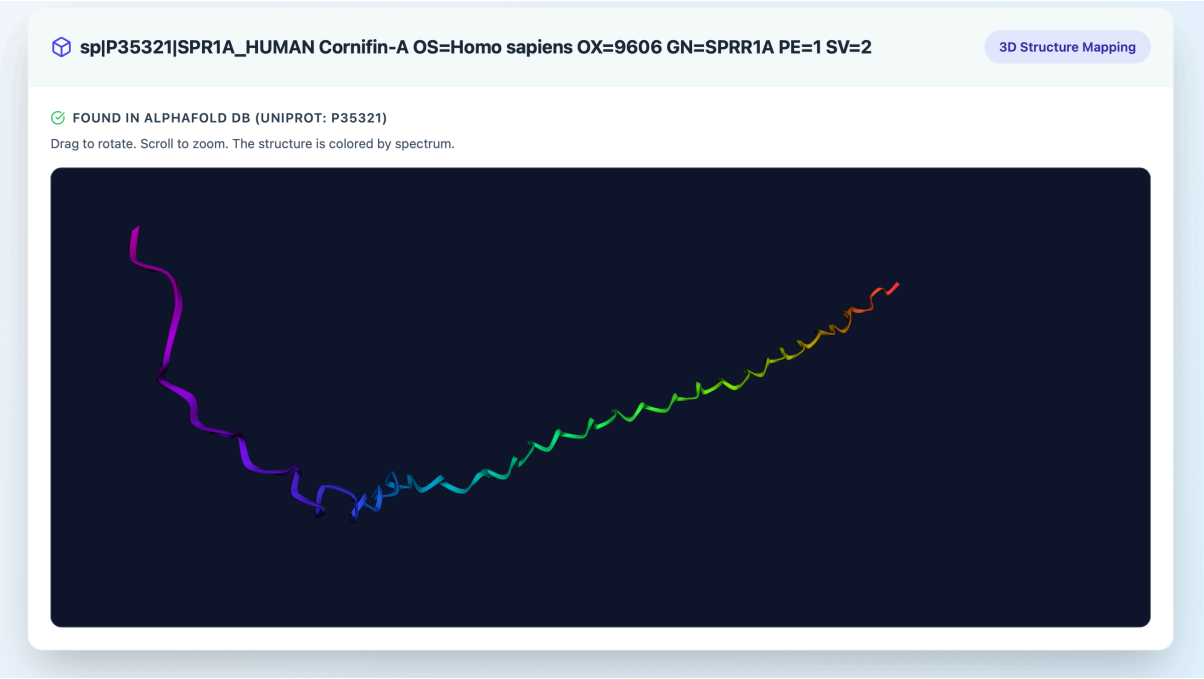

Figure 5.10: 3D structure visualization screen. The platform retrieves known structures from AlphaFold DB or falls back to ESMFold for sequence-based prediction when necessary.

Appendix C - UML Use Case Diagram

This appendix presents a UML use case diagram derived from the functional requirements described in Chapter 3. In UML, use cases are represented as ellipses inside the system boundary, while actors and external services are placed outside the boundary. Non-functional requirements are treated as quality constraints that support these behaviours rather than as separate use cases.

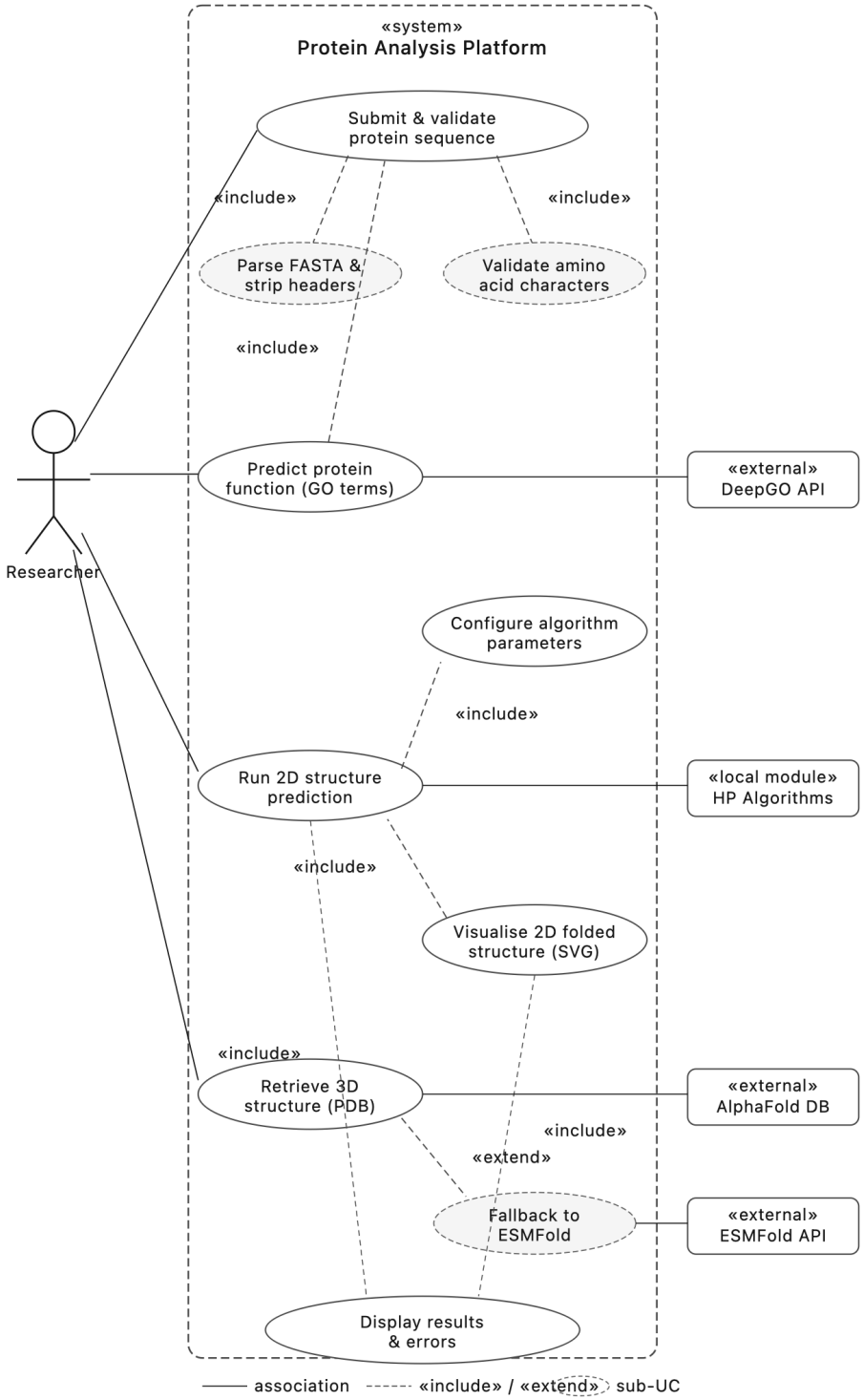

Figure 5.11: UML use case diagram of the web platform. The diagram summarizes the main functional requirements of the platform, including FASTA input validation, GO-term prediction, 2D HP structure prediction, SVG visualization, 3D structure retrieval, and result/error display.

Instituto Superior de Engenharia
Politécnico de Coimbra